KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Tholurpatti (Po), Thottiam (Tk), Trichy (Dt)-621 215.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

20AD502 – R PROGRAMMING

YEAR/SEMESTER: 03 / 05

HANDOUT NOTES

ACADEMIC YEAR 2023-2024

UNIT 1: INTRODUCTION

1.1 Overview of R

"R is an interpreted computer programming language which was created by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand." The **R Development Core Team** currently develops R. It is also a software environment used to analyze **statistical information, graphical representation, reporting**, and **data modeling**. R is the implementation of the **S programming language**, which is combined with **lexical scoping semantics.**

Features of R Programming

R is a domain-specific programming language which aims to do data analysis. It has some unique features which make it very powerful. The most important arguably being the notation of vectors. These vectors allow us to perform a complex operation on a set of values in a single command. There are the following features of R programming:

1. It is a simple and effective programming language which has been well developed.
2. It is data analysis software.
3. It is a well-designed, easy, and effective language which has the concepts of user-defined, looping, conditional, and various I/O facilities.
4. It has a consistent and incorporated set of tools which are used for data analysis.
5. For different types of calculation on arrays, lists and vectors, R contains a suite of operators.
6. It provides effective data handling and storage facility.
7. It is an open-source, powerful, and highly extensible software.
8. It provides highly extensible graphical techniques.
9. It allows us to perform multiple calculations using vectors.
10. R is an interpreted language.

Programming Features of R R Packages:

One of the major features of R is it has a wide availability of libraries. R has CRAN(Comprehensive R Archive Network), which is a repository holding more than 10, 0000 packages.

Distributed Computing:

Distributed computing is a model in which components of a software system are shared among multiple computers to improve efficiency and performance. Two new packages **ddR and multidplyr** used for distributed programming in R were released in November 2015.

1.2 R data types and objects

In programming languages, we need to use various variables to store various information. Variables are the reserved memory location to store values. As we create a variable in our program, some space is reserved in memory. In R, there are several data types such as integer, string, etc. The operating system allocates memory based on the data type of the variable and decides what can be stored in the reserved memory. In contrast to other programming languages like C and java in R, the variables are not declared as some data type. The variables are assigned with R-Objects and the data type of the R-object becomes the data type of the variable. There are many types of R-objects. The frequently used ones are −

Vectors

Lists

Matrices

Arrays

Factors

Data Frames

In R programming, the very basic data types are the R-objects called **vectors** which hold elements of different classes as shown above. Please note in R the number of classes is not confined to only the above six types. For example, we can use many atomic vectors and create an array whose class will become array.

Vectors:

When you want to create vector with more than one element, you should use c() function which means to combine the elements into a vector.

```
# Create a vector.
apple <- c('red','green',"yellow")
print(apple)

# Get the class of the vector.
print(class(apple))
```

Lists:

A list is an R-object which can contain many different types of elements inside it like vectors, functions and even another list inside it.

```
# Create a list.
list1 <- list(c(2,5,3),21.3,sin)

# Print the list.
print(list1)
```

Matrices:

A matrix is a two-dimensional rectangular data set. It can be created using a vector input to the matrix function.

```
# Create a matrix.
M = matrix( c('a','a','b','c','b','a'), nrow = 2, ncol = 3, byrow = TRUE)
print(M)
```

When we execute the above code, it produces the following result −

```
     [,1] [,2] [,3]
[1,] "a"  "a"  "b"
[2,] "c"  "b"  "a"
```

Array:

While matrices are confined to two dimensions, arrays can be of any number of dimensions. The array function takes a dim attribute which creates the required number of dimension. In the below example we create an array with two elements which are 3x3 matrices each.

```
# Create an array.
a <- array(c('green','yellow'),dim = c(3,3,2))
print(a)
```

Factors:

Factors are the r-objects which are created using a vector. It stores the vector along with the distinct values of the elements in the vector as labels. The labels are always character irrespective of whether it is numeric or character or Boolean etc. in the input vector. They are useful in statistical modeling.

Data Frames

Data frames are tabular data objects. Unlike a matrix in data frame each column can contain different modes of data. The first column can be numeric while the second column can be character and third column can be logical. It is a list of vectors of equal length.

Data Frames are created using the **data.frame ()** function.

1.3 Reading and Writing Data

Functions for Reading Data into R

There are a few very useful functions for reading data into R.

1. **read.table() and read.csv()** are two popular functions used for reading tabular data into R.
2. **readLines()** is used for reading lines from a text file.
3. **source()** is a very useful function for reading in R code files from a another R program.
4. **dget()** function is also used for reading in R code files.
5. **load()** function is used for reading in saved workspaces
6. **unserialize()** function is used for reading single R objects in binary format.

Writing data files with write.table()

Following are few important arguments usually used in write.table() function.

1. x, the object to be written, typically a data frame
2. file, the name of the file which the data are to be written to
3. sep, the field separator string
4. col.names, a logical value indicating whether the column names of x are to be written along with x, or a character vector of column names to be written

5. row.names, a logical value indicating whether the row names of x are to be written along with x, or a character vector of row names to be written
6. na, the string to use for missing values in the data.

1.4 Subsetting R Objects

In R Programming Language, subsetting allows the user to access elements from an object. It takes out a portion from the object based on the condition provided. There are 4 ways of subsetting in R programming. Each of the methods depends on the usability of the user and the type of object. For example, if there is a dataframe with many columns such as states, country, and population and suppose the user wants to extract states from it, then subsetting is used to do this operation. In this article, let us discuss the implementation of different types of subsetting in R programming.

1.5 Installation of R:

R programming is a very popular language and to work on that we have to install two things, i.e., R and RStudio. R and RStudio works together to create a project on R.Installing R to the local computer is very easy. First, we must know which operating system we are using so that we can download it accordingly.The official site https://cloud.r-project.org provides binary files for major operating systems including Windows, Linux, and Mac OS. In some Linux distributions, R is installed by default, which we can verify from the console by entering R.

To install R, either we can get it from the site https://cloud.r-project.org or can use commands from the terminal.

1.6 Running R:

1) Go to the official site of R programming
2) Click on the CRAN link on the left sidebar
3) Select a mirror
4) Click “Download R for Windows”
5) Click on the link that downloads the base distribution
6) Run the file and follow the steps in the instructions to install R.

1.7 Packages of R:

R packages are the collection of R functions, sample data, and compile codes. In the R environment, these packages are stored under a directory called "library." During installation, R installs a set of packages. We can add packages later when they are needed for some specific purpose. Only the default

packages will be available when we start the R console. Other packages which are already installed will be loaded explicitly to be used by the R program.

There is the following list of commands to be used to check, verify, and use the R package.

COMPLEX NUMBER IN R:

Numbers in R can be divided into 3 different categories: • Numeric: It represents both whole and floating-point numbers. For example, 123, 32.43, etc. • Integer: It represents only whole numbers and is denoted by L. For example, 23L, 39L, etc. • Complex: It represents complex numbers with imaginary parts. The imaginary parts are denoted by i. For example, 2 + 3i, 5i, etc.

ROUNDING:

Round function in R, rounds off the values in its first argument to the specified number of decimal places. Round() function in R rounds off the list of values in vector and also rounds off the column of a dataframe. It can also accomplished using signif() function. Let see an example of each.

• Round() function to round off the values of a vector.

• round off the values of vector using signif() function in R.

• round off a column in R dataframe using round() function.

• round off the values of column in dataframe using signif() function

MODULO AND INTEGER QUOTIENTS

Modulo:

Modulus Operation is an arithmetic operation in R which calculates the remainder after division of two numeric variables. This recipe demonstrates how to carry out the Modulus operation using two numeric variables while storing them in a third variable.

Step 1:

Creating two numeric variables We assign numbers to two variables

a = 10 b = 4

Step 2:

Multiplying the two variables We use the arithmetic operator " %% " to carry out this task and finally store the result in a third variable

storing the result of the modulus arithmetic operation of the two numbers stored in variables 'a' and 'b' in 'result'

result = a %% b

displaying the value stored in result

result

2

VARIABLE NAMES AND ASSIGNMENT

A variable provides us with named storage that our programs can manipulate. A variable in R can store an atomic vector, group of atomic vectors or a combination of many R objects. A valid variable name consists of letters, numbers and the dot or underline characters. The variable name starts with a letter or the dot not followed by a number.

Variable Name	Validity	Reason
var_name2.	valid	Has letters, numbers, dot and underscore
var_name%	Invalid	Has the character '%'. Only dot(.) and underscore allowed.
2var_name	invalid	Starts with a number
.var_name, var.name	valid	Can start with a dot(.) but the dot(.)should not be followed by a number.
.2var_name	invalid	The starting dot is followed by a number making it invalid.
_var_name	invalid	Starts with _ which is not valid

OPERATORS:

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. R language is rich in built-in operators and provides following types of operators.

Types of Operators

• Arithmetic Operators

• Relational Operators

• Logical Operators

• Assignment Operators

• Miscellaneous Operators

INTEGERS:

In order to create an integer variable in R, we invoke the integer function. We can be assured that y is indeed an integer by applying the is.integer function.

Incidentally, we can coerce a numeric value into an integer with the as.integer function.

FACTORS

Factors are the data objects which are used to categorize the data and store it as levels. They can store both strings and integers. They are useful in the columns which have a limited number of unique values. Like "Male, "Female" and True, False etc. They are useful in data analysis for statistical modeling.

LOGICAL OPERATIONS:

Following table shows the logical operators supported by R language. It is applicable only to vectors of type logical, numeric or complex. All numbers greater than 1 are considered as logical value TRUE. Each element of the first vector is compared with the corresponding element of the second vector. The result of comparison is a Boolean value.

The logical operator && and || considers only the first element of the vectors and give a vector of single element as output.

&&	Called Logical AND operator. Takes first element of both the vectors and gives the TRUE only if both are TRUE.	`v <- c(3,0,TRUE,2+2i)`
		`t <- c(1,3,TRUE,2+3i)`
		`print(v&&t)`
		it produces the following result:
		`[1] TRUE`

||	Called Logical OR operator. Takes first element of both the vectors and gives the TRUE only if both are TRUE.	`v <- c(0,0,TRUE,2+2i)`
		`t <- c(0,3,TRUE,2+3i)`
		`print(v||t)`
		it produces the following result:
		`[1] FALSE`

UNIT 2

CONTROL STRUCTURES AND VECTORS

2.1 CONTROL STRUCTURES

Control statements are expressions used to control the execution and flow of the program based on the conditions provided in the statements. In R, there are decision-making structures like if-else that control execution of the program conditionally. There are also looping structures that loop or repeat code sections based on certain conditions and state. These structures are used to make a decision after assessing the variable.

In R programming, there are 8 types of control statements as follows:

- if condition
- if-else condition
- for loop
- nested loops
- while loop
- repeat and break statement
- return statement
- next statement

1. if

The if-else in R enforce conditional execution of code. They are an important part of R's decision-making capability. It allows us to make a decision based on the result of a condition. The if statement contains a condition that evaluates to a logical output

CODE

```
if(a>b){
  print("a is greater than b")
} else{
  print("b is greater than a")
}
```

2. ifelse() Function

The ifelse() function acts like the if-else structure. The following is the syntax of the ifelse() function in R:

ifelse(condition, exp_if_true, exp_if_false)

3. switch

The switch is an easier way to choose between multiple alternatives than multiple if-else statements. The R switch takes a single input argument and executes a particular code based on the value of the input. Each possible value of the input is called a case.

4. for loops

The for loop in R, repeats through sequences to perform repeated tasks. They work with an iterable variable to go through a sequence. The following is the syntax of for loops in R:

5. while Loops

The while loop in R evaluates a condition. If the condition evaluates to TRUE it loops through a code block, whereas if the condition evaluates to FALSE it exits the loop. The while loop in R keeps looping through the enclosed code block as long as the condition is TRUE. This can also result in an infinite loop sometimes which is something to avoid.

6.break Statement

The break statement can break out of a loop. Imagine a loop searching a specific element in a sequence. The loop needs to keep going until either it finds the element or until the end of the sequence. If it finds the element early, further looping is not needed. In such a case, the R break statement can “break” us out of the loop early.

8. repeat loop

The repeat loop in R initiates an infinite loop from the get-go. The only way to get out of the loop is to use the break statement. The repeat loop is useful when you don’t know the required number of iterations.

2.2 Function

A function is a set of statements organized together to perform a specific task. R has a large number of in-built functions and the user can create their own functions.

2.2.1 Function Definition

An R function is created by using the keyword **function**. The basic syntax of an R function definition is as follows −

```
function_name <- function(arg_1, arg_2, ...) {
  Function body
}
```

Function Components

The different parts of a function are −

- **Function Name** − This is the actual name of the function. It is stored in R environment as an object with this name.
- **Arguments** − An argument is a placeholder. When a function is invoked, you pass a value to the argument. Arguments are optional; that is, a function may contain no arguments. Also arguments can have default values.
- **Function Body** − The function body contains a collection of statements that defines what the function does.
- **Return Value** − The return value of a function is the last expression in the function body to be evaluated.

R has many **in-built** functions which can be directly called in the program without defining them first. We can also create and use our own functions referred as **user defined** functions.

Lazy Evaluation of Function

Arguments to functions are evaluated lazily, which means so they are evaluated only when needed by the function body.

```
# Create a function with arguments.
new.function <- function(a, b) {
  print(a^2)
  print(a)
  print(b)
}

# Evaluate the function without supplying one of the arguments.
new.function(6)
```

When we execute the above code, it produces the following result −

```
[1] 36
[1] 6
Error in print(b) : argument "b" is missing, with no default
```

2.3 Scoping Rules

The scoping rules of a language determine how a value is associated with a *free variable* in a function. R uses *lexical scoping* or *static scoping*. An alternative to lexical

scoping is *dynamic scoping* which is implemented by some languages. Lexical scoping turns out to be particularly useful for simplifying statistical computations.

The scoping rules of a language determine how values are assigned to free variables. Free variables are not formal arguments and are not local variables (assigned insided the function body).

what is an environment?

An *environment* is a collection of (symbol, value) pairs, i.e. x is a symbol and 3.14 might be its value. Every environment has a parent environment and it is possible for an environment to have multiple "children". The only environment without a parent is the *empty environment*.

2.3.1Lexical Scoping: Why Does It Matter?

Typically, a function is defined in the global environment, so that the values of free variables are just found in the user's workspace. This behavior is logical for most people and is usually the "right thing" to do. However, in R you can have functions defined *inside other functions* (languages like C don't let you do this). Now things get interesting—in this case the environment in which a function is defined is the body of another function!

Here is an example of a function that returns another function as its return value. Remember, in R functions are treated like any other object and so this is perfectly valid.

2.3.2 Lexical vs. Dynamic Scoping

With lexical scoping the value of y in the function g is looked up in the environment in which the function was defined, in this case the global environment, so the value of y is 10. With dynamic scoping, the value of y is looked up in the environment from which the function was *called* (sometimes referred to as the *calling environment*). In R the calling environment is known as the *parent frame*. In this case, the value of y would be 2.

When a function is *defined* in the global environment and is subsequently *called* from the global environment, then the defining environment and the calling environment are the same. This can sometimes give the appearance of dynamic scoping.

2.4 Dates And Times

Dates

R has developed a special representation for dates and times. Dates are represented by the Date class and times are represented by the POSIXct or the POSIXlt class. Dates are stored internally as the number of days since 1970-01-01 while times are stored internally as the number of seconds since 1970-01-01.

Times

Times are represented by the POSIXct or the POSIXlt class. POSIXct is just a very large integer under the hood. It use a useful class when you want to store times in something like a data frame. POSIXlt is a list underneath and it stores a bunch of other useful information like the day of the week, day of the year, month, day of the month. This is useful when you need that kind of information.

There are a number of generic functions that work on dates and times to help you extract pieces of dates and/or times.

2.5 Introduction to Functions

2.5.1Preview Of Some Important R Data Structures

A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks. Data structures in R programming are tools for holding multiple values. R's base data structures are often organized by their dimensionality (1D, 2D, or nD) and whether they're homogeneous (all elements must be of the identical type) or heterogeneous (the elements are often of various types). This gives rise to the six data types which are most frequently utilized in data analysis.

The most essential data structures used in R include:

- **Vectors**
- **Lists**
- **Dataframes**
- **Matrices**
- **Arrays**
- **Factors**

One of the key features of R is that it can handle complex statistical operations in an easy and optimized way. R handles complex computations using: Vector – A basic data structure of R containing the same type of data Matrices – A matrix is a rectangular array of numbers or other mathematical objects. We can do operations such as addition and multiplication on Matrix in R. Lists – Lists store collections of objects when vectors are of same type and length in a matrix. Data Frames – Generated by combining together multiple vectors such that each vector becomes a separate column

2.6 Vectors in R

In R, **Vector** is a basic data structure in R that contains element of similar type. These data types in R can be logical, integer, double, character, complex or raw.

In R using the function, **typeof()** one can check the data type of vector.

One more significant property of R vector is its length. The function **length()** determines the number of elements in the vector.

Adding and Deleting Vector Elements

Vectors are stored like arrays in C, contiguously, and thus you cannot insert or delete elements—something you may be used to if you are a Python programmer. The size of a vector is determined at its creation, so if you wish to add or delete elements.

Obtaining the Length of a Vector

We can obtain the length of a vector by using the length () function:

```
> x <- c(1,2,4)
> length(x)
[1] 3
```

2.7 Characters Strings in R

Character strings are another common data type, used to represent text.In R, character strings (or simply "strings") are indicated by double quotation marks. To create a string, just enter text between two paris of these quotes.

Most characters can be used in a string, with a couple of exceptions, one being the backslash character, "\". This character is called the **escape character** and is used to insert characters that would otherwise be difficult to add. For example, without an escape character, adding a double quote *inside* a string would pose a problem, as R would assume that you meant the string to end upon seeing the double quote. With an escape character, however, adding a double quote inside your string is easy, you simply prepend the double

quote with the backslash. The table below shows some of the other characters that can be "escaped" in this way.

2.8 Matrices in R

Matrices are Data frames which contain lists of homogeneous data in a tabular format. We can perform arithmetic operations on some elements of the matrix or the whole matrix itself in R. Matrices are special cases of a more general R type of object: arrays. Arrays can be multidimensional. For example, a three-dimensional array would consist of rows, columns, and layers, not just rows and columns as in the matrix case.

escape sequence	result
\n	Starts a new line
\t	Horizontal tab
\b	Backspace
\"	Double quote
\\	Single Backslash

If one needs to convert a numerical value to a string, one can use the "paste" function, as shown below: > paste(5) [1] "5" This function can also be used to concatenate corresponding elements of vectors containing strings, as shown here:

```
> paste(c("A","B","C"),
c("1","2","3"))
 [1] "A 1" "B 2" "C 3"
```

Notice in the example above, a single space was inserted between each letter and number. If one prefers to concatenate the elements with no space between them, one can use the paste0() function instead, as the following suggests. > paste0(c("A","B","C"),c("1","2","3")) [1] "A1" "B2" "C3"

Matrices

can represent the binding of two or more vectors of equal length. Analogous operations can be used to change the size of a matrix. For instance, the rbind() (row bind) and cbind() (column bind) functions let you add rows or columns to a matrix

Applying Functions to Matrix Rows and Columns

One of the most famous and most used features of R is the *apply() family of functions, such as apply(), tapply(), and lapply(). Here, we'll look at apply(), which instructs R to call a user-specified function on each of the rows or each of the columns of a matrix.

Using the apply() Function

This is the general form of apply for matrices:

```
apply(m,dimcode,f,fargs)
```

where the arguments are as follows:

- `m` is the matrix.
- `dimcode` is the dimension, equal to 1 if the function applies to rows or 2 for columns.
- `f` is the function to be applied.
- `fargs` is an optional set of arguments to be supplied to `f`.

2.9 Lists in R

Lists are R Data Types stores collections of objects of differing lengths and types using **list()** function. In contrast to a vector, in which all elements must be of the same mode, R's list structure can combine objects of different types.. The list plays a central role in R, forming the basis for data frames, object-oriented programming, and so on.

Creating Lists

Technically, a list is a vector. Ordinary vectors—those of the type we've been using so far in this book—are termed atomic vectors, since their components cannot be broken down into smaller components. In contrast, lists are referred to as recursive vectors.

Let's consider an employee database. For each employee, we wish to store the name, salary, and a Boolean indicating union membership. Since we have three different modes here—character, numeric, and logical—it's a perfect place for using lists. Our entire database might then be a list of lists, or some other kind of list such as a data frame, though we won't pursue that here.

2.10 Data Frames

The sequence and number of observations in the vectors must be the same for each vector in the Data Frame to represent a **DataSet.**

The first, second and third entries in each vector, for example, must represent the observations collected from first, second and third sampling units respectively.

Programming in R

There are several built-in functions library and add-on tools available for R and they continue to grow at an incredible rate. Yet programs need performing a task for which no functions exist. Since **R** is itself a programming language, extending its functionality to accommodate more procedures depends on the complexity of the procedure and the level of R proficiency of the user.

2.11 Classes Vectors: Generating sequences

sequence() function in R Language is used to create a vector of sequenced elements. It creates vectors with specified length, and specified differences between elements. It is similar to seq() function.

Syntax: sequence(x)

Parameters:
x: Maximum element of vector

2.13 Extracting elements of a vector using subscripts
To extract (also known as indexing or subscripting) one or more values (more generally known as elements) from a vector we use the square bracket `[ ]` notation. The general approach is to name the object you wish to extract from, then a set of square brackets with an index of the element you wish to extract contained within the square brackets. This index can be a position or the result of a logical test.

Positional index
To extract elements based on their position we simply write the position inside the `[ ]`. For example, to extract the 3rd value of `my_vec`

Logical index
Another really useful way to extract data from a vector is to use a logical expression as an index. For example, to extract all elements with a value greater than 4 in the vector `my_vec`

```
my_vec[my_vec > 4]
## [1] 6 7
```

Here, the logical expression is `my_vec > 4` and R will only extract those elements that satisfy this logical condition. So how does this actually work? If we look at the output of just the logical expression without the square brackets you can see that R returns a vector containing either `TRUE` or `FALSE` which correspond to whether the logical condition is

```
my_vec > 4
## [1] FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE
```

satisfied for each element. In this case only the 4th and 8th elements return a `TRUE` as their value is greater than 4.

Programming in R There are several built-in functions library and add-on tools available for R and they continue to grow at an incredible rate. Yet programs need performing a task for which no functions exist. Since R is itself a programming language, extending its functionality to accommodate more procedures depends on the complexity of the procedure and the level of R proficiency of the user. User Created Functions: Expressions – Command entered at R command prompt. Assignment – Assigns name to an object. Arithmetic Operations – When numeric values are there, we use arithmetic operations to perform operations.

Vectors and subscript

The vector type is really the heart of R. It's hard to imagine R code, or even an interactive R session, that doesn't involve vectors. The elements of a vector must all have the same mode, or data type. We can have a vector consisting of three character strings (of mode character) or three integer elements (of mode integer), but not a vector with one integer element and two character string elements. Vectors in R are the same as the arrays in C language which are used to hold multiple data values of the same type. One major key point is that in R the indexing of the vector will start from '1' and not from '0'. We can create numeric vectors and character vectors as well.

Recycling - The automatic lengthening of vectors in certain settings

Filtering - The extraction of subsets of vectors

Vectorization-Where functions are applied element-wise to vectors

2.14 Working with logical subscripts

When you subscript with a logical vector, you are selecting the elements that correspond to TRUE.

That is, the logical vector doing the subscripting is the same length as the original vector, and it is the result of some comparison operation. A logical subscript is similar to a negative number subscript. They both leave the elements of the result in the same order as the original with some of the elements not there.

2.15 Scalars - Vectors - Arrays - and Matrices

Four common object types that store data are:

1. **Scalars**: store a single numeric value.
2. **Strings**: store a set of one or more characters.
3. **Vectors**: store several scalar or string elements.

4. **Data Frames**. Store several vectors (meaning that they contain several rows and columns).

Scalars

A scalar data structure is the most basic data type that holds only a single atomic value at a time. Using scalars, more complex data types can be constructed.

Vectors

A vector object is just a combination of several scalars stored as a single object. For example, the numbers from one to ten could be a vector of length 10, and the characters in the English alphabet could be a vector of length 26. Like scalars, vectors can be either numeric or character (but not both!).

Matrices

Matrices are special cases of a more general R type of object: arrays. Arrays can be multidimensional. For example, a three-dimensional array would consist of rows, columns, and layers, not just rows and columns as in the matrix case.Since we specified the matrix entries in the preceding example, and there were four of them, we did not need to specify both ncol and nrow; just nrow or ncol would have been enough.

Extended Example: Generating a Covariance Matrix

This example demonstrates R's row() and col() functions, whose arguments are matrices. For example, for a matrix a, row(a[2,8]) will return the row number of that element of a, which is 2. Well, we knew row(a[2,8]) is in row

Let's consider an example. When writing simulation code for multivariate normal distributions—for instance, using mvrnorm() from the MASS library—we need to specify a covariance matrix. The key point for our purposes here is that the matrix is symmetric; for example, the element in row 1, column 2 is equal to the element in row 2, column 1.

Arrays

Arrays are the R data objects which can store data in more than two dimensions. For example − If we create an array of dimension (2, 3, 4) then it creates 4 rectangular matrices each with 2 rows and 3 columns. Arrays can store only data type.An array is created using the array() function. It takes vectors as input and uses the values in the dim parameter to create an array.

2.16 Adding and Deleting Vector Elements

Vectors are one-dimensional arrays that can hold numeric data, character data, or logical data. The combine function c() or colon : is used to form the vector.

2.18 Matrices and Arrays as Vectors

Arrays and matrices (and even lists, in a sense) are actually vectors too. They merely have extra class attributes. For example, matrices have the number of rows and columns.

```
> m
  [,1] [,2]
[1,]    1    2
[2,]    3    4
> m + 10:13
  [,1] [,2]
[1,]   11   14
[2,]   14   17
```

The 2-by-2 matrix m is stored as a four-element vector, column-wise, as (1,3,2,4). We then added (10,11,12,13) to it, yielding (11,14,14,17), but R remembered that we were working with matrices and thus gave the 2-by-2 result you see in the example.

Recycling

When applying an operation to two vectors that requires them to be the same length, R automatically recycles, or repeats, the shorter one, until it is long enough to match the longer one. Here is an example:

2.19 Arithmetic Operations

The basic arithmetic operations can all be performed on multi-dimensional arrays, and act on the arrays element-by-element. For example,the runtime scales linearly with the number of elements in the multi-dimensional array, because the arithmetic operation is performed on each individual index. For example, the runtime for adding a pair of M×N matrices scales as (O(MN)).

2.20 The logical Operation

The most commonly-used function for array multiplication is the dot function, which takes two array inputs x and y and returns their "dot product". It constructs a product by summing over the last index of array x, and over the next-to-last index of array y (or over its last index, if y is a 1D array). This may sound like a complicated rule, but you should be able to convince yourself that it corresponds to the appropriate type of multiplication operation for the most common cases encountered in linear algebra:

2.21VECTOR INDEXING:

Vector elements are accessed using indexing vectors, which can be numeric, character or logical vectors.You can access an individual element of a vector by its position (or "index"), indicated using square brackets. In R, the first element has an index of 1. You can access multiple elements of a vector by specifying a vector of element ***indices inside the square brackets***. All the methods that we learned about in the last section can be used to generate these indexing vectors.

2.22 Common Operations on Vectors

Vectors are the most basic data types in R. Even a single object created is also stored in the form of a vector. Vectors are nothing but arrays as defined in other languages. Vectors contain a sequence of homogeneous types of data. If mixed values are given then it auto converts the data according to the precedence. There are various operations that can be performed on vectors in R.

1. Combining Vector in R

Functions are used to combine vectors. In order to combine the two vectors in R, we will create two new vectors ‘n’ and ‘s’. Then, we will create another vector that will combine these two using c(n,s) as follows:

For example:

```
> #Author DataFlair
> n = c(1, 2, 3, 4)
> s = c("Hadoop", "Spark", "HIVE", "Flink")
> c(n,s)
```

2. Arithmetic Operations on Vectors in R

Arithmetic operations on vectors can be performed member-by-member.

For example:

Suppose we have two vectors a and b:

```
> #Author DataFlair
> a = c (1, 3)
> b = c (1, 3)
> a + b #Addition
```

For subtraction:

```
> a - b #Subtraction
```

For division:

```
> a / b #Division
```

For remainder operation:

```
> a %% b #Remainder Operation
```

3. Logical Index Vector in R

By using a logical index vector in R, we can form a new vector from a given vector, which has the same length as the original vector. If the corresponding members of the original vector are included in the slice, then vector members are TRUE and otherwise FALSE.

For example:

```
> #Author DataFlair
> S = c("bb", "cc")
> L = c(TRUE, TRUE) #Defining our Logical Vector
> S[L] #This will return elements of vector S that corrospond to logic vector L
```

*3. **Numeric Index***

For indexing a numerical value in R, we specify the index between square braces [ ]. If our index is negative, then R will return us all the values except for the index that we have specified. For example, specifying [-2] will prompt R to convert -2 into its absolute value and then search for the value that occupies that index.

5. Duplicate Index

The index vector allows duplicate values. Hence, the following retrieves a member twice in one operation.

For example:

```
> # Author DataFlair
> s = c("aa", "bb", "cc", "dd", "ee")
> s[c(2,3,3)]
```

6. **Range Indexes**

To produce a vector slice between two indexes, we can use the colon operator "**:**". It is convenient for situations involving large vectors.

For example:

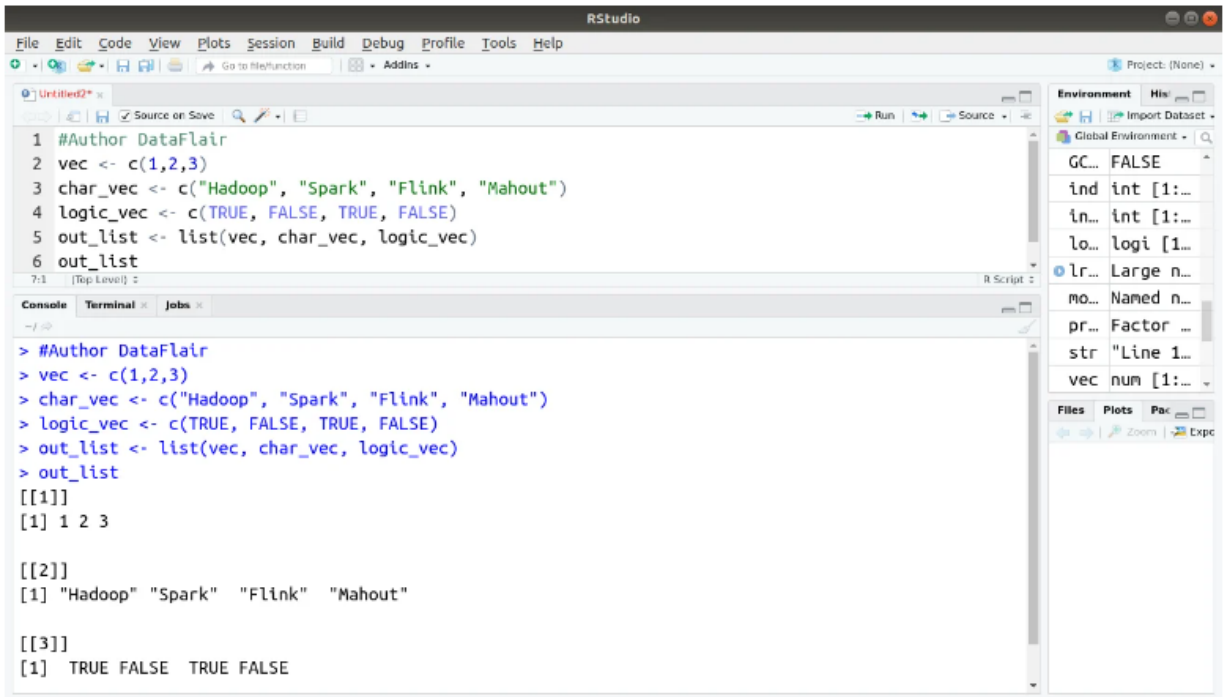
```
> # Author DataFlair
> s = c("aa", "bb", "cc", "dd", "ee")
> s[1:3]
```

UNIT III

LISTS

List:

R list is the object which contains elements of different types – like *strings, numbers, vectors and another list* inside it. R list can also contain a matrix or a function as its elements. A list is a vector but with heterogeneous data elements. A list in R is created with the use of **list()** function. R allows accessing elements of a list with the use of the index value. In R, the indexing of a list starts with 1 instead of 0 like other programming languages.



Creating a List:

How to Create Lists in R Programming?

The process of creating a list is the same as a vector. In R, the vector is created with the help of **c()** function. Like **c()** function, there is another function, i.e., **list()** which is used to create a list in R. A list avoid the drawback of the vector which is data type. We can add the elements in the list of different data types.

Syntax

list()

Example 2: Creating the list with different data type

```
list_data<-list("Shubham","Arpita",c(1,2,3,4,5),TRUE,FALSE,22.5,12L)
print(list_data)
```

In the above example, the list function will create a list with character, logical, numeric, and vector element. It will give the following output.

Giving a name to list elements

R provides a very easy way for accessing elements, i.e., by giving the name to each element of a list. By assigning names to the elements, we can access the element easily. There are only three steps to print the list data corresponding to the name:

1. Creating a list.
2. Assign a name to the list elements with the help of names() function.

General List Operations:

We know that a list is a data structure that is used to store data in a linear fashion and supports the elements of the multiple data types. In R programming, we will do all the following operations that are performed on a list.

List indexing

You can access a list component in several different ways:

```
> j$salary
[1] 55000
> j[["salary"]]
[1] 55000
> j[[2]]
 [1] 55000
```

We can access the values in the list using the index positions. To access the single element, we can directly specify the index position.

We can refer to list components by their numerical indices, treating the list as a vector. However, note that in this case, we use double brackets instead of single ones.

Syntax:

list_object[index]

Where: list_object is the list and index specify index position.

We access the following:

1. First element from the second list, i.e. mango – 1st element
2. First element from the third list, i.e. guava – 1st element
3. Second element from the first list, i.e. apples – 2nd element

It is also possible to access the elements from the nested list object by specifying the list names through the $ operator.

Syntax:

list_object$list_name

With the previous scenario, we can also get a particular element from the nested list through the index position.

Syntax:
list_object$list_name[index]
Example:

Get the second elements from the apples and mangoes nested lists.

Adding and Deleting List Elements

The operations of adding and deleting list elements arise in a surprising number of contexts. This is especially true for data structures in which lists form the foundation, such as data frames and R classes. New components can be added after a list is created.

Add Element to List at Specified Position

Use the after parameter to specify the position where you wanted to add the element. The following example adds elements after the first position.

Add Multiple Elements to List

Package rlist provides a list.append() function to add multiple elements to the list in R. In order to use this function first, you need to install R package by using install.packages("rlist") and load it using the library("rlist").

Delete

It is possible to delete an entire list by assigning it to NULL.

Syntax:

list_object =NULL

Where: list_object is the list

Example:

Delete the apples nested list.

```
#create list for fruits
fruits=list(apples=list(34,45),mangoes=list(14,35),guava=list(12,34))

#delete apples nested list
fruits$apples=NULL

print(fruits)
```

Result:

```
$mangoes
$mangoes[[1]]
[1] 14

$mangoes[[2]]
[1] 35


$guava
$guava[[1]]
[1] 12

$guava[[2]]
[1] 34
```

Getting the Size of a List

In R programming language, to find the length of every elements in a list, **the function lengths()** can be used.

Syntax:

lengths(x).

Extended Example: Text Concordance

We will write a function called **findwords()** that will determine which words are in a text file and compile a list of the locations of each word's occurrences in the text. This would be useful for contextual analysis,

Inputs

- Corpus: A collection of documents.

Outputs

- Selected Documents: Documents containing the queried word.
- Concordances: A table of concordances.

Concordance finds the queried word in a text and displays the context in which this word is used. Results in a single color come from the same document. The widget

can output selected documents for further analysis or a table of concordances for the queried word.

Note that the widget finds only exact matches of a word, which means that if you query the word ‘do’, the word ‘doctor’ won’t appear in the results.

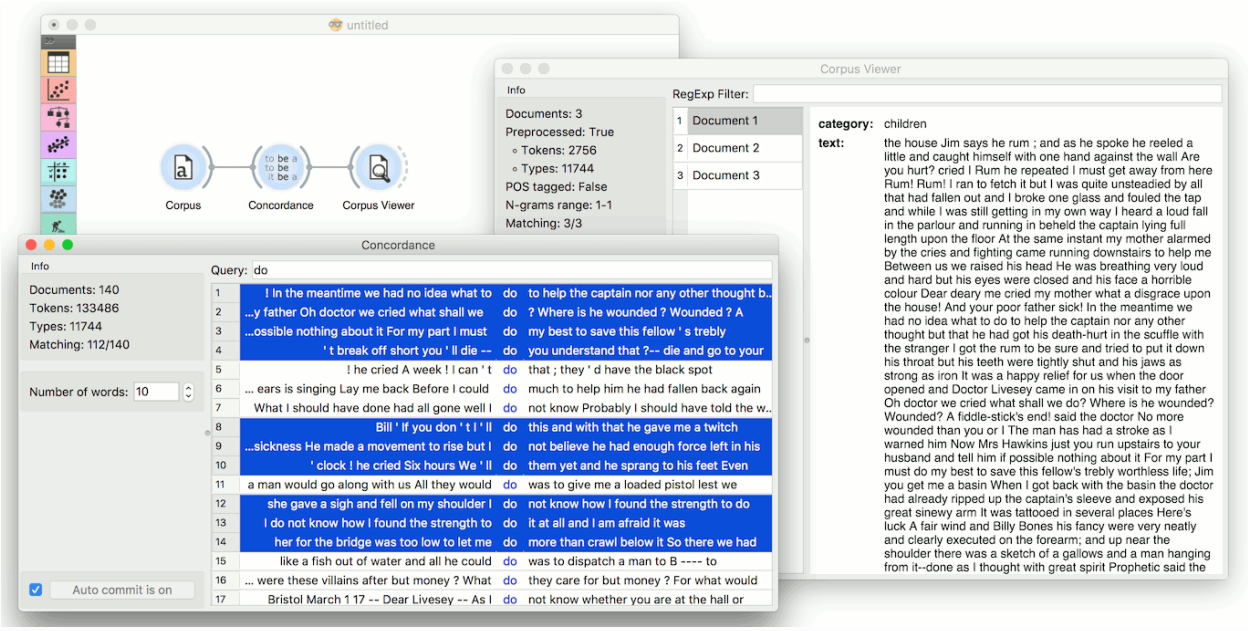
1. *Information*:
   - *Documents*: number of documents on the input.
   - *Tokens*: number of tokens on the input.
   - *Types*: number of unique tokens on the input.
   - *Matching*: number of documents containing the queried word.
2. *Number of words*: the number of words displayed on each side of the queried word.
3. Queried word.
4. If *Auto commit is on*, selected documents are communicated automatically. Alternatively press *Commit*.

Examples

Concordance can be used for displaying word contexts in a corpus. First, we load *book-excerpts.tab* in *Corpus*. Then we connect **Corpus** to **Concordance** and search for concordances of a word ‘doctor’. The widget displays all documents containing the word ‘doctor’ together with their surrounding (contextual) words.

Now we can select those documents that contain interesting contexts and output them to *Corpus Viewer* to inspect them further.

https://orange3-text.readthedocs.io/en/latest/_images/Concordance-Example1.png



In the second example, we will output concordances instead. We will keep the *book-excerpts.tab* in Corpus and the connection to **Concordance**. Our queried word remains 'doctor'.

This time, we will connect **Data Table** to **Concordance** and select Concordances output instead. In the **Data Table**, we get a list of concordances for the queried word and the corresponding documents. Now, we will save this table with **Save Data** widget, so we can use it in other projects or for further analysis.

Data Frames

A data frame is a table or a two-dimensional array-like structure in which each column contains values of one variable and each row contains one set of values from each column.

Following are the characteristics of a data frame.

- The column names should be non-empty.
- The row names should be unique.
- The data stored in a data frame can be of numeric, factor or character type.

- Each column should contain same number of data items.

A data frame is like a matrix, with a two-dimensional rows-andcolumns structure. However, it differs from a matrix in that each column may have a different mode. For instance, one column may consist of numbers, and another column might have character strings. In this sense, just as lists are the heterogeneous analogs of vectors in one dimension, data frames are the heterogeneous analogs of matrices for two-dimensional data.

Creating Data Frames

Use the data.frame() function to create a data frame:

```
# Create a data frame
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)

# Print the data frame
Data_Frame
```

Access Item

We can use single brackets `[ ]`, double brackets `[[ ]]` or `$` to access columns from a data frame:

When we execute the above code, it produces the following result −

```
Training
1 Strength
2  Stamina
3    Other
[1] Strength Stamina  Other
Levels: Other Stamina Strength
[1] Strength Stamina  Other
Levels: Other Stamina Strength
```

Other Matrix-Like Operation

Various matrix operations also apply to data frames. Most notably and usefully, we can do filtering to extract various subdata frames of interest.

Extracting Subdata Frames

As mentioned, a data frame can be viewed in row-and-column terms. In particular, we can extract subdata frames by rows or columns. Here's an example:

```
> examsquiz[2:5,]
  Exam.1 Exam.2 Quiz
2    3.3      2  3.7
3    4.0      4  4.0
4    2.3      0  3.3
5    2.3      1  3.3
> examsquiz[2:5,2]
[1] 2 4 0 1
> class(examsquiz[2:5,2])
[1] "numeric"
> examsquiz[2:5,2,drop=FALSE]
  Exam.2
2      2
3      4
4      0
5      1
> class(examsquiz[2:5,2,drop=FALSE])
[1] "data.frame"
```

Note that in that second call, since examsquiz[2:5,2] is a vector, R created a vector instead of another data frame. We can also do filtering. Here's how to extract the subframe of all students whose first exam score was at least 3.8:

```
> examsquiz[examsquiz$Exam.1 >= 3.8,]
```

	Exam.1	Exam.2	Quiz
3	4	4.0	4.0

More on Treatment of NA Values

As the name indicates, Missing values are those elements which are not known. NA or NaN are reserved words that indicate a missing value in R Programming language for q arithmetical operations that are undefined.

R – handling Missing Values

Missing values are practical in life. For example, some cells in spreadsheets are empty. If an insensible or impossible arithmetic operation is tried then NAs occur.

Suppose the second exam score for the first student had been missing. Then we would have typed the following into that line when we were preparing the data file:

```
2.0   NA   4.0
```

In any subsequent statistical analyses, R would do its best to cope with the missing data. However, in some situations, we need to set the option na.rm=TRUE (Remove the NA values), explicitly telling R to ignore NA values. For instance, with the missing exam score, calculating the mean score on exam 2 by calling R's mean() function would skip that first student in finding the mean. Otherwise, R would just report NA for the mean.

Syntax: function(vector,na.rm)

Where,

- vector is input vector
- na.rm is to remove NA values
- function is to perform operation on vector like sum ,mean ,min ,max etc

na.rm in dataframe

We have to use apply function to apply the function on the dataframe with na.rm function

Syntax: apply(dataframe, 2, function, na.rm)

Where

- dataframe is the input dataframe
- function is to perform some operations like mean,min ,max etc
- 2 represents column
- na.rm is to remove NA values

Example :

```
# create a dataframe with 3 columns
data = data.frame(column1 = c(1,2,NA,34),
                  column2 = c(NA,34,56,NA),
                  column3 = c(NA,NA,32,56))

# display
print(data)

# calculate mean including  NA values
apply(data, 2, mean , na.rm = FALSE)

# calculate sum including  NA values
apply(data, 2, sum , na.rm = FALSE)

# calculate min including  NA values
apply(data, 2, min, na.rm = FALSE)

# calculate max including  NA values
apply(data, 2, max , na.rm = FALSE)


# calculate standard deviation including
# NA values
apply(data, 2, sd, na.rm = FALSE)
```

Output:

```
  column1 column2 column3
1       1      NA      NA
2       2      34      NA
3      NA      56      32
4      34      NA      56
column1 column2 column3
     NA      NA      NA
column1 column2 column3
     NA      NA      NA
column1 column2 column3
     NA      NA      NA
column1 column2 column3
     NA      NA      NA
column1 column2 column3
     NA      NA      NA
```

Using the rbind() and cbind() Functions and Alternative

The rbind() and cbind() matrix functions work with data frames, too, providing that you have compatible sizes, of course. For instance, you can use cbind() to add a new column that has the same length as the existing columns.

rbind:

The binding or combining of the rows is very easy with the rbind() function in R. rbind() stands for row binding. In simpler terms joining of multiple rows to form a

single batch. It may include joining two data frames, vectors, and more. **To binding or combining the rows of two different data frames should be in a same or equal length.**

Syntax of the rbind() function

rbind(): The rbind or the row bind function is used to bind or combine the multiple group of rows together.

```
rbind(x,x1)
```

Copy

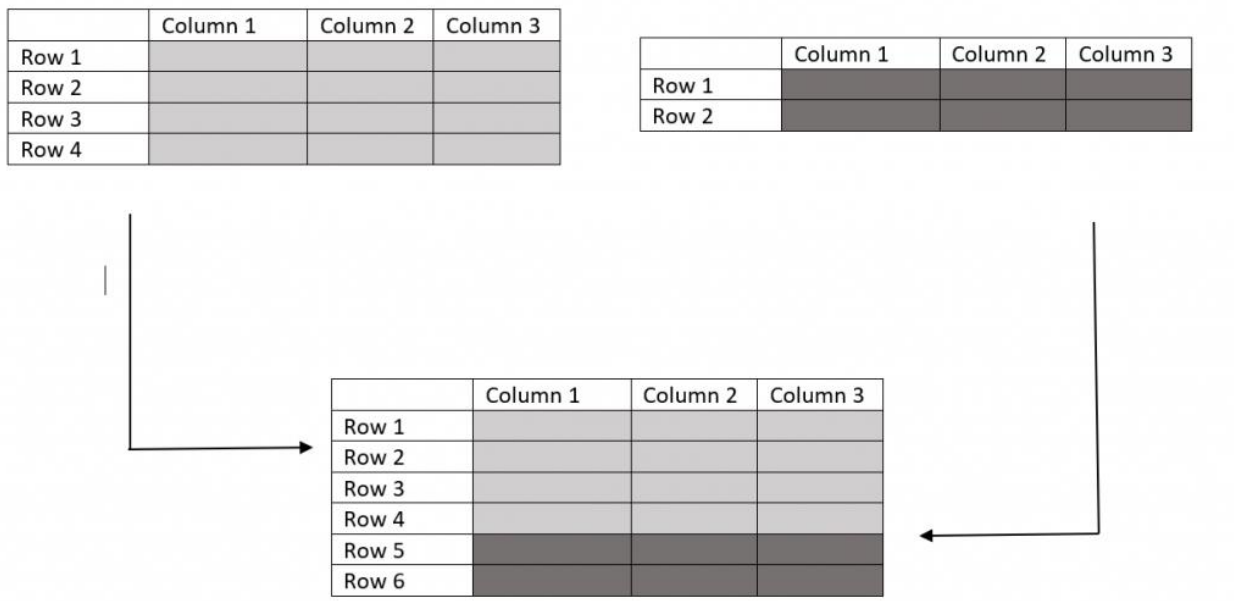
Where:

- **X** = the input data.
- **X1** = The data need to be binded.

The idea of binding rows using rbind()

The idea of binding or combing the rows of multiple data frames is highly beneficial in data manipulation.The below diagram will definitely get you the idea of working the rbind() function.

You can see that how rows of different data frames will bound/combined by the rbind() function from the below diagram.



Use the rbind() function to add new rows in a Data Frame:

```
Data_Frame<-data.frame(
Training=c("Strength","Stamina","Other"),
Pulse=c(100,150,120),
Duration=c(60,30,45)
)
#Add a new row
New_row_DF<-rbind(Data_Frame,c("Strength",110,110))
#Print the new row
New_row_DF
```

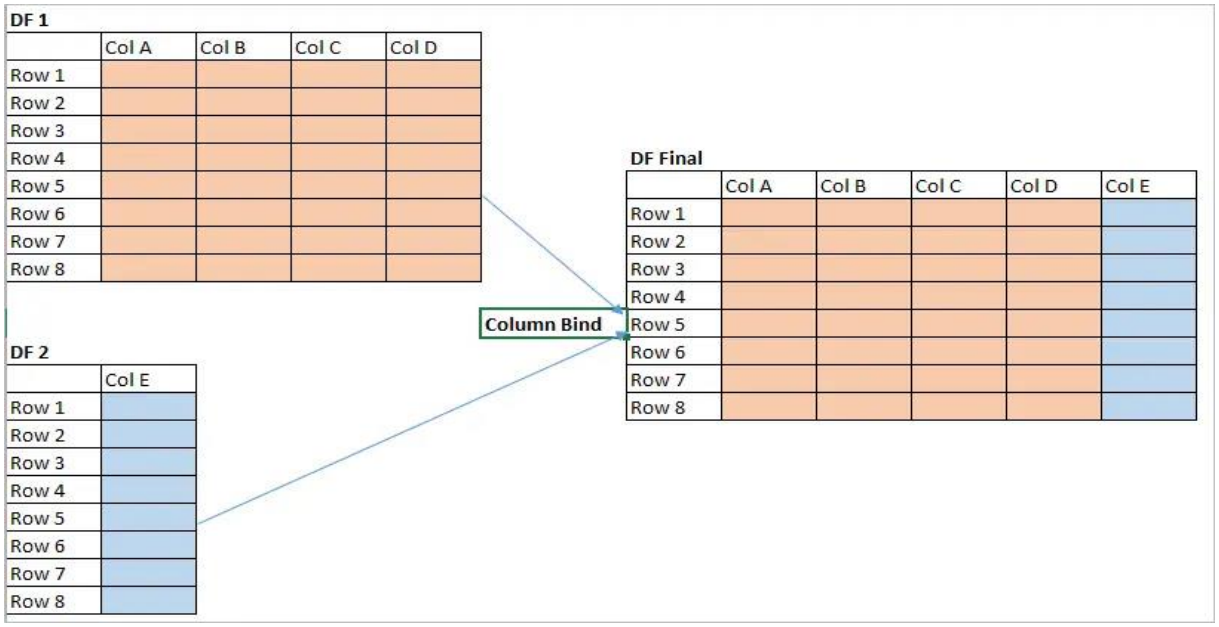
When we execute the above code, it produces the following result −

```
Training Pulse Duration
1 Strength   100       60
2  Stamina   150       30
3    Other   120       45
4 Strength   110      110
```

Cbind:

Cbind in R appends or combines vector, matrix or data frame by columns. cbind() function in R appends or joins, two or more dataframes in column wise. same column bind operation can also be performed using bind_cols() function of the dplyr package. Lets see column bind in R which emphasizes on bind_cols() function and cbind() function with an example for each.

Cbind () — column bind function is used for merging two data frames together given that the number of rows in both the data frames are equal. cbind can append vectors, matrices or any data frame by columns. This recipe demonstrates an example using cbind.



Note : The number of rows in two dataframes needs to be same for both cbind() function and bind_cols() function.

Syntax for cbind() in R:

cbind(x1,x2)

x1,x2 can be data frame, matrix or vector.

Step 1- Define two dataframes

```
df1 <- data.frame(name = c('A','B','C','D','E','F'), age = c(22,25,28,19,15,23))
print(df1)
```

"df1 is":

```
  name age
1    A  22
2    B  25
3    C  28
4    D  19
5    E  15
6    F  23
```

```
df2 <- data.frame(gender = c('Male','Male','Female','Male','Female','Female'))
print(df2)
```

"df2 is":

gender

1 Male

2 Male

3 Female

4 Male

5 Female

6 Female

Step 2 - Apply cbind()

final_data <- cbind(df1,df2) print(final_data)

"Output of code is":

name age gender

1 A 22 Male

2 B 25 Male

3 C 28 Female

4 D 19 Male

5 E 15 Female

6 F 23 Female

Use the cbind() function to add new columns in a Data Frame:

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)

# Add a new column
New_col_DF <- cbind(Data_Frame, Steps = c(1000, 6000, 2000))

# Print the new column
New_col_DF
```

When we execute the above code, it produces the following result −

```
Training Pulse Duration Steps
1 Strength   100       60  1000
2  Stamina   150       30  6000
3    Other   120       45  2000
```

Column Bind in R using bind_cols() function of Dplyr()

Syntax for bind_cols() in R:

bind_cols(x1,x2)

x1,x2 are the data frames

Now, bind_cols() function of dplyr, takes two dataframes df1 and df2 as argument and the results are appended or column binded to a data frame as shown below. The number of rows in two dataframes needs to be same for bind_cols() function.

```
# bind_cols in R: column bind the data frames.

library(dplyr)

colbinded_df = bind_cols(df1,df2)

colbinded_df
```

so the resultant column bind data frame by using bind_cols() function will be cbind() function and bind_cols() Function performs in the similar manner and can be used alternatively for column binding. For Further understanding on bind_cols() function refer r dplyr package document.

UNIT 4

FACTORS AND TABLES

Factors are used to represent categorical data and can be unordered or ordered. One can think of a factor as an integer vector where each integer has a label. Factors are important in statistical modeling and are treated specially by modelling functions like **lm()** and **glm()**. Using factors with labels is better than using integers because factors are self-describing. Having a variable that has values “Male” and “Female” is better than a variable that has values 1 and 2. Factor objects can be created with the factor() function.

```
> x <- factor(c("yes", "yes", "no", "yes", "no"))
> x
 [1] yes yes no yes no
 Levels: no yes
 > table(x)
 X
 no yes
2 3
> ## See the underlying representation of factor
 > unclass(x)
[1] 2 2 1 2 1
 attr(,"levels")
 [1] "no" "yes"
```

Often factors will be automatically created for you when you read a dataset in using a function like read.table(). Those functions often default to creating factors when they encounter data that look like characters or strings. The order of the levels of a factor can be set using the levels argument to factor(). This can be important in linear modelling because the first level is used as the baseline level.

```
 > x <- factor(c("yes", "yes", "no", "yes", "no"))
> x ## Levels are put in alphabetical order
 [1] yes yes no yes no
Levels: no yes
```

```
> x <- factor(c("yes", "yes", "no", "yes", "no"),
+ levels = c("yes", "no"))
> x
[1] yes yes no yes no
Levels: yes no
```

4.1 Factors and Levels

An R factor might be viewed simply as a vector with a bit more information added (though, as seen below, it's different from this internally). That extra information consists of a record of the distinct values in that vector, called levels.

```
> x <- c(5,12,13,12)
 > xf <- factor(x)
> xf
 [1] 5 12 13 12
Levels: 5 12 13
```

The distinct values in xf—5, 12, and 13—are the levels here. Let's take a look inside

```
> str(xf)
Factor w/ 3 levels "5","12","13":1232
> unclass(xf)
[1] 1 2 3 2
attr(,"levels")
[1] "5" "12" "13"
```

4.1.1 Common Functions Used with Factors

With factors, we have yet another member of the family of apply functions, tapply. We'll look at that function, as well as two other functions commonly used with factors: split() and by().

The tapply() Function

Tapply() is used to apply a function over subsets of a vector. It is primarily used when we have the following circumstances:

1. A dataset that can be broken up into groups (via categorical variables - aka factors)
2. We desire to break the dataset up into groups

3. Within each group, we want to apply a function

The arguments to tapply() are as follows:

- x is a vector
- INDEX is a factor or a list of factors (or else they are coerced to factors)
- FUN is a function to be applied
- ... contains other arguments to be passed FUN

```
# syntax of tapply function
tapply(x, INDEX, FUN, ..., simplify = TRUE)
```

To provide an example we'll use the built in mtcars dataset and calculate the mean of the mpg variable grouped by the cyl variable.

```
# show first few rows of mtcars
head(mtcars)
##                    mpg cyl disp  hp drat    wt  qsec vs am
gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1
4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1
4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1
4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0
3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0
3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0
3    1

# get the mean of the mpg column grouped by cylinders
tapply(mtcars$mpg, mtcars$cyl, mean)
##        4        6        8
## 26.66364 19.74286 15.10000
```

In typical usage, the call tapply(x,f,g) has x as a vector, f as a factor or list of factors, and g as a function. The function g() in our little example above would be R's built-in mean() function. If we wanted to group by both party and another factor, say gender, we would need f to consist of the two factors, party and gender.

Each factor in f must have the same length as x. This makes sense in light of the voter example above; we should have as many party affiliations as ages. If a

```
> ages <- c(25,26,55,37,21,42)
> affils <- c("R","D","D","R","U","D")
```

```
> tapply(ages,affils,mean)
D R U
41 31 21
```

Let's look at what happened. The function tapply() treated the vector ("R","D","D","R","U","D") as a factor with levels "D", "R", and "U". It noted that "D" occurred in indices 2, 3 and 6; "R" occurred in indices 1 and 4; and "U" occurred in index 5. For convenience, let's refer to the three index vectors (2,3,6), (1,4), and (5) as x, y, and z, respectively. Then tapply() computed mean(u[x]), mean(u[y]), and mean(u[z]) and returned those means in a three-element vector. And that vector's element names are "D", "R", and "U", reflecting the factor levels that were used by tapply().

The split() Function

In contrast to tapply(), which splits a vector into groups and then applies a specified function on each group, split() stops at that first stage, just forming the groups.

The basic form, without bells and whistles, is split(x,f), with x and f playing roles similar to those in the call tapply(x,f,g); that is, x being a vector or data frame and f being a factor or a list of factors. The action is to split x 124 Chapter 6 into groups, which are returned in a list. (Note that x is allowed to be a data frame with split() but not with tapply().)

```
> d
  gender age income over25
1      M  47  55000      1
2      M  59  88000      1
3      F  21  32450      0
4      M  32  76500      1
5      F  33 123000      1
6      F  24  45650      0
> split(d$income,list(d$gender,d$over25))
$F.0
[1] 32450 45650

$M.0
numeric(0)

$F.1
[1] 123000

$M.1
[1] 55000 88000 76500
```

The output of split() is a list, and recall that list components are denoted by dollar signs. So the last vector, for example, was named "M.1" to indicate that it was the result of combining "M" in the first factor and 1 in the second.

The vector g, taken as a factor, has three levels: "M", "F", and "I". The indices corresponding to the first level are 1, 5, and 6, which means that g[1], g[5], and g[6] all have the value "M". So, R sets the M component of the output to elements 1, 5, and 6 of 1:7, which is the vector (1,5,6).

We can take a similar approach to simplify the code in our text concordance example from Section 4.2.4. There, we wished to input a text file, determine which words were in the text, and then output a list giving the words and their locations within the text. We can use split() to make short work of writing the code, as follows:

The call to scan() returns a list txt of the words read in from the file tf. So, txt[[1]] will contain the first word input from the file, txt[[2]] will contain the second word, and so on; length(txt) will thus be the total number of words read.

Meanwhile, txt itself, as the second argument in split() above, will be taken as a factor. The levels of that factor will be the various words in the file. If, for instance, the file contains the word world 6 times and climate was there 10 times, then “world” and “climate” will be two of the levels of txt. The call to split() will then determine where these and the other words appear in txt.

The by() Function

by() function in R applies a function to specified subsets of a data frame.First parameter of by() function, takes up the data and second parameter is by which the function is applied and third parameter is the function.

Syntax of by() function in R:

by(data, data$byvar, FUN)

data	an **R** object, normally a data frame, possibly a matrix.
data$byvar	a factor or a list of factors by which the function is applied
FUN	a function to be applied to the subsets of data.

The by() function can be used here. It works like tapply() (which it calls internally, in fact), but it is applied to objects rather than vectors. Here's how to use it for the desired regression analyses:

```
> aba <- read.csv("abalone.data",header=TRUE)
 > by(aba,aba$Gender,function(m) lm(m[,2]~m[,3]))
aba$Gender: F
Call:
lm(formula = m[, 2] ~ m[, 3])
Coefficients:
(Intercept) m[, 3]
0.04288 1.17918
 ------------------------------------------------------------
aba$Gender: I
Call:
 lm(formula = m[, 2] ~ m[, 3])
Coefficients:
(Intercept) m[, 3]
 0.02997 1.21833
 ------------------------------------------------------------
 aba$Gender:
 M Call:
lm(formula = m[, 2] ~ m[, 3])
 Coefficients:
 (Intercept) m[, 3]
0.03653 1.19480
```

Calls to by() look very similar to calls to tapply(), with the first argument specifying our data, the second the grouping factor, and the third the function to be applied to each group. Just as tapply() forms groups of indices of a vector according to levels of a factor, this by() call finds groups of row numbers of the data frame aba.

That creates three subdata frames: one for each gender level of M, F, and I. The anonymous function we defined regresses the second column of its matrix argument m against the third column.

4.2 WORKING WITH TABLES

Tables are often essential for organzing and summarizing your data, especially with categorical variables. When creating a table in R, it considers your table as a specifc type of object (called “table”) which is very similar to a data frame.

To begin exploring R tables, consider this example:

```
> u <- c(22,8,33,6,8,29,-2)
> fl <- list(c(5,12,13,12,13,5,13),c("a","bc","a","a","bc","a","a"))
> tapply(u,fl,length)
a bc
5 2 NA
12 1 1
13 2 1
```

Here, tapply() again temporarily breaks into subvectors, and then applies the length() function to each subvector. (Note that this is independent of what’s in u. Our focus now is purely on the factors.) Those subvector lengths are the counts of the occurrences of each of the 3 × 2=6 combinations of the two factors. For instance, 5 occurred twice with "a" and not at all with "bc"; hence the entries 2 and NA in the first row of the output. In statistics, this is called a contingency table.

The first argument in a call to table() is either a factor or a list of factors. The two factors here were (5,12,13,12,13,5,13) and ("a","bc","a","a","bc", "a","a"). In this case, an object that is interpretable as a factor is counted as one.

Typically a data frame serves as the table() data argument. Suppose for instance the file ct.dat consists of election-polling data, in which candidate X is running for reelection. The ct.dat file looks like this:

```
"Vote for X" "Voted For X Last Time"
"Yes"  "Yes"
```

```
 "Yes"  "No"
 "No"  "No"
"Not Sure"  "Yes"
 "No"  "No"
```

In the usual statistical fashion, each row in this file represents one subject under study. In this case, we have asked five people the following two questions:

This gives us five rows in the data file. Let's read in the file:

```
> ct <- read.table("ct.dat",header=T)
> ct
     Vote.for.X Voted.for.X.Last.Time
1   Yes                      Yes
2    Yes                      No
3    No                       No
4    Not Sure                 Yes
5    No                       No
```

We can use the table() function to compute the contingency table for this data:

```
> cttab <- table(ct)
> cttab
                Voted.for.X.Last.Time
Vote.for.X      No      Yes
    No           2       0
  Not Sure       0       1
      Yes        1       1
```

for example, two people who said "no" to the first and second questions. The 1 in the middle-right indicates that one person answered "not sure" to the first question and "yes" to the second question. We can also get one-dimensional counts, which are counts on a single factor, as follows

```
> table(c(5,12,13,12,8,5))
 5  8  12  13
 2 1   2   1
```

Here's an example of a three-dimensional table, involving voters' genders, race (white, black, Asian, and other), and political views (liberal or conservative)

```
> v # the data frame
 gender race pol
1 M  W  L
2 M  W  L
3 F  A  C
4 M  O  L
5 F  B  L
6 F  B  C
> vt <- table(v)
> vt
, , pol = C
        race
gender  A  B  O  W
   F    1  1  0  0
   M    0  0  0  0
, , pol = L
        race
gender  A  B  O  W
   F    0  1  0  0
   M    0  0  1  2
```

R prints out a three-dimensional table as a series of two-dimensional tables. In this case, it generates a table of gender and race for conservatives and then a corresponding table for liberals. For example, the second twodimensional table says that there were two white male liberals

4.2.1 MATRIX/ARRAY-LIKE OPERATIONS ON TABLES

The matrix/array operations can be used on data frames, they can be applied to tables, too.

For example, we can access the table cell counts using matrix notation. Let's apply this to our voting example from the previous section.

```
> class(cttab)
[1] "table"
> cttab[1,1]
[1] 2
> cttab[1,]
 No Yes
  2   0
```

In the second command, even though the first command had shown that cttab had class **"cttab",** we treated it as a matrix and printed out its "[1,1] element." Continuing this idea, the third command printed the first column of this "matrix." We can multiply the matrix by a scalar. For instance, here's how to change cell counts to proportions:

```
> ctt/5
          Voted.for.X.Last.Time
Vote.for.X  No Yes
  No       0.4 0.0
  Not Sure 0.0 0.2
  Yes      0.2 0.2
```

In statistics, the marginal values of a variable are those obtained when this variable is held constant while others are summed. In the voting example, the marginal values of the Vote.for.X variable are 2 + 0 = 2, 0 + 1 = 1, and 1 + 1 = 2. We can of course obtain these via the matrix apply() function:

```
> apply(ctt,1,sum)
      No Not Sure      Yes
       2        1        2
```

Note that the labels here, such as No, came from the row names of the matrix, which table() produced. But R supplies a function addmargins() for this purpose that is, to find marginal totals. Here's an example:

```
> addmargins(cttab)
          Voted.for.X.Last.Time
Vote.for.X No Yes Sum
  No        2   0   2
  Not Sure  0   1   1
  Yes       1   1   2
  Sum       3   2   5
```

4.2.2 Extracting a Sub Table

V1	V2	V3
A	B	1
A	C	1
A	D	0
A	E	1
A	F	0
A	G	0
A	H	0

Here, extract a subtable data2 and keep all line where V3 == 1 like:

V1	V2	V3
A	B	1
A	C	1
A	E	1

It checks all v3 data which have 1 then it extracted to display in an another subtable.

4.2.3 Finding the Largest Cells in a Table

It can be difficult to view a table that is very big, with a large number of rows or dimensions. One approach might be to focus on the cells with the largest frequencies. That's the purpose of the tabdom() function developed below it reports the dominant frequencies in a table. Here's a simple call:

tabdom(tbl,k)

The function tells us that the values 5 and 12 were the most frequent in d, with four instances each, and the next most frequent value was 4, with two instances.

As another example, consider our table cttab in the examples in the preceding sections:

```
> tabdom(cttab,2)

     Vote.for.X        Voted.For.X.Last.Time    Freq

1        No                    No                 2

3        Yes                   No                 1
```

So the combination No-No was most frequent, with two instances, with the second most frequent being Yes-No, with one instance.

Well, how is this accomplished? It looks fairly complicated, but actually the work is made pretty easy by a trick, exploiting the fact that you can present tables in data frame format. Let's use our cttab table again.

```
> as.data.frame(cttab)
  Vote.for.X Voted.For.X.Last.Time Freq
1         No                    No    2
2   Not Sure                    No    0
3        Yes                    No    1
4         No                   Yes    0
5   Not Sure                   Yes    1
6        Yes                   Yes    1
```

Note that this is not the original data frame ct from which the table cttab was constructed. It is simply a different presentation of the table itself. There is one row for each combination of the factors, with a Freq column added to show the number of instances of each combination. This latter feature makes our task quite easy.

```
1  # finds the cells in table tbl with the k highest frequencies; handling
2  # of ties is unrefined
3  tabdom <- function(tbl,k) {
4     # create a data frame representation of tbl, adding a Freq column
5     tbldf <- as.data.frame(tbl)
6     # determine the proper positions of the frequencies in a sorted order
7     freqord <- order(tbldf$Freq,decreasing=TRUE)
8     # rearrange the data frame in that order, and take the first k rows
9     dom <- tbldf[freqord,][1:k,]
10    return(dom)
11 }
```

The sorting approach in line 7, which makes use of order(), is the standard way to sort a data frame. The approach taken here converting a table to a data frame.

4.3 Math Functions

R contains built-in functions for the math operations and, of course, for statistical distributions.

R includes an extensive set of built-in math functions. Here is a partial list:

- exp(): Exponential function, base e
- log(): Natural logarithm
- log10(): Logarithm base 10
- sqrt(): Square root
- abs(): Absolute value
- sin(), cos(), and so on: Trig functions
- min() and max(): Minimum value and maximum value within a vector
- which.min() and which.max(): Index of the minimal element and maximal element of a vector
- pmin() and pmax(): Element-wise minima and maxima of several vectors
- sum() and prod(): Sum and product of the elements of a vector
- cumsum() and cumprod(): Cumulative sum and product of the elements of a vector

- round(), floor(), and ceiling(): Round to the closest integer, to the closest integer below, and to the closest integer above
- factorial(): Factorial function.

4.3.1 Calculating a Probability

Calculating a probability using the prod() function. Suppose we have n independent events, and the i th event has the probability pi of occurring. What is the probability of exactly one of these events occurring?

Suppose first that n = 3 and our events are named A, B, and C. Then we break down the computation as follows:

P(exactly one event occurs) = P(A and not B and not C) + P(not A and B and not C) + P(not A and not B and C)

P(A and not B and not C) would be pA(1 − pB)(1 − pC), and so on. For general n, that is calculated as follows:

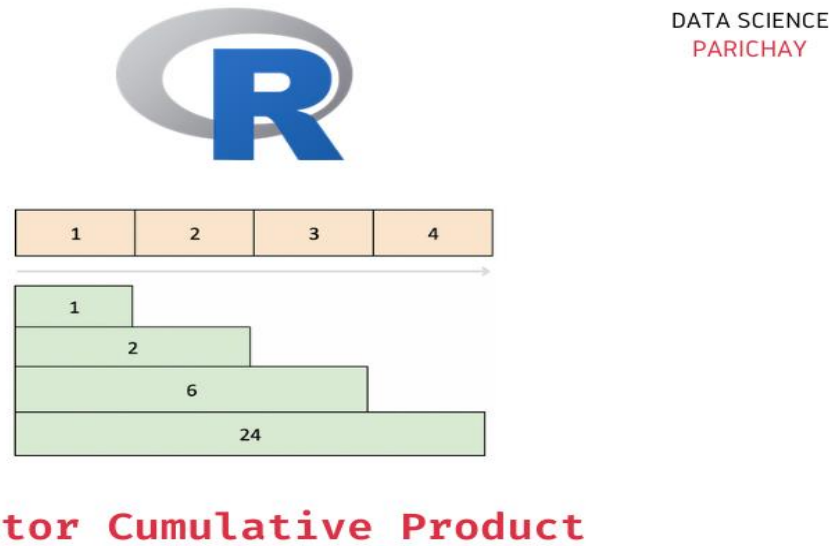
$$\sum_{i=1}^{n} p_i(1-p_1)\ldots(1-p_{i-1})(1-p_{i+1})\ldots(1-p_n)$$

(The i th term inside the sum is the probability that event i occurs and all the others do not occur.)

4.3.2 Cumulative Sums and Products

The functions cumsum() and cumprod() return cumulative sums and products.

Cumulative Products



cumprod() function in R Language is used to calculate the cumulative product of the vector passed as argument.

Syntax: *cumprod(x)*

Parameters:

x: *Numeric Object*

Example 1:

```
# R program to illustrate
# the use of cumprod() Function
# Calling cumprod() Function
cumprod(1:4)
cumprod(-1:-6)
```

Output:

```
[1]  1  2  6 24[1]   -1    2   -6   24 -120  720
```

Cumulative sum

The **cumsum() function** in R computes the cumulative sum of elements in a vector object.

Table 3

	value	group	cs
1	1	A	1
2	2	A	3
3	3	A	6
4	4	B	4
5	5	B	9
6	6	B	15
7	7	C	7
8	8	C	15
9	9	C	24

Syntax

The syntax for the cumsum() function

```
cumsum(x)
```

Example

```
# implementing the cumsum() function to take the sum of elements in the vector objecs
cumsum(1:10)
cumsum(c(2, 3, 1, -4, 2))
```

- **Line 2**: We obtain the cumulative sum of numbers starting from 1 to 10 using the cumsum() function.
- **Line 3**: We obtain the cumulative sum of random numbers of a vector object using the cumsum() function.

4.3.3 Minima and Maxima

In R, we can find the minimum or maximum value of a vector or data frame.

We use the min() and max() function to find minimum and maximum value respectively.

- The min() function returns the minimum value of a vector or data frame.
- The max() function returns the maximum value of a vector or data frame.

Syntax of min() and max() in R

The syntax of the min() and max() function is

For min()

```
min(collection, na.rm = Boolean)
```

For max()

```
max(collection, na.rm = Boolean)
```

In both the syntax,

- collection - is a vector or data frame
- na.rm (optional) - is a boolean value that indicates whether value should be kept or removed,

There is quite a difference between min() and pmin(). The former simply combines all its arguments into one long vector and returns the minimum value in that vector. In contrast, if pmin() is applied to two or more vectors, it returns a vector of the pair-wise minima, hence the name pmin. Here's an example:

Example 1: Use of min() in R

```
numbers <- c(2,4,6,8,10)

# return minimum value present in numbers

min(numbers)  # 2

characters <- c("s", "a", "p", "b")

# return alphabetically minimum value in characters

min(characters)  # "a"
```

Output

```
[1] 2
[1] "a"
```

Here,

```
numbers <- c(2, NA, 6, 7, NA, 10)

# return smallest value

min(numbers, na.rm = TRUE)  # 2
```

Output

```
[1] 2
```

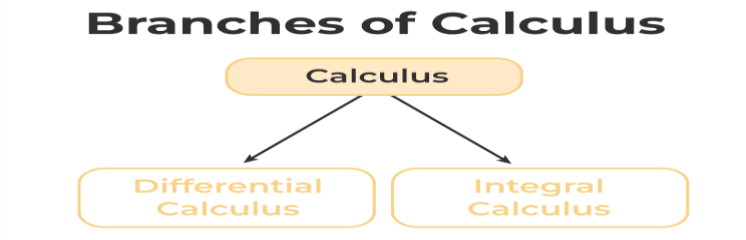
Here, we have used the `na.rm` argument to handle `NA` values.
By setting `na.rm` to `TRUE`, we have removed `NA` before the computation. So the output will be **2** not `NA`.

> **Note**: Similar to `min()`, we can use `max()` with NA values too.

4.3.4 Calculus

Calculus is a branch of mathematics that involves the study of rates of change. Before calculus was invented, all math was static: It could only help calculate objects that were perfectly still.

Calculus is a subset of mathematics concerned with the study of continuous transition. Calculus is also known as infinitesimal calculus or "infinite calculus." The analysis of continuous change of functions is known as classical calculus. Derivatives and integrals are the two most important ideas of calculus. The integral is the measure of the region under the curve, while the derivative is the measure of the rate of change of a function. The integral accumulates the discrete values of a function over a number of values, while the derivative describes the function at a given point. Two types of calculus are,



Differential Calculus

Differential Calculus deals with the issues of determining the rate of change of a parameter with respect to other variables. Derivatives are used to find the maxima and minima values of a function in order to find the best solution. The analysis of the boundary of a quotient leads to differential calculus. It is concerned with variables such as x and y, functions f(x), and the resulting variations in x and y. Differentials are represented by the symbols dy and dx. Differentiation refers to the method of determining derivatives. A function's derivative is defined by dy/dx or f' (x). It denotes that the equation is the derivative of y with respect to x.

In R programming, derivative of a function can be computed using **deriv()** and **D()** function. It is used to compute derivatives of simple expressions.

Syntax:

deriv(expr,name)

D(expr, name)

Parameters:

expr: represents an expression or a formula with no LHS

name: represents character vector to which derivatives will be computed

Example:

Expression or formula

f = expression(x^2 + 5*x + 1)

Derivative

cat("Using deriv() function:\n")

print(deriv(f, "x"))

cat("\nUsing D() function:\n")

print(D(f, 'x')

Output:

```
Using deriv() function:
expression({
    .value <- x^2 + 5 * x + 1
    .grad <- array(0, c(length(.value), 1L), list(NULL, c("x")))
    .grad[, "x"] <- 2 * x + 5
    attr(.value, "gradient") <- .grad
    .value
})

Using D() function:
2 * x + 5
```

```r
# Little harder derivative

# Using deriv() Function
cat("Using deriv() function:\n")
print(deriv(quote(sinpi(x^2)), "x"))

# Using D() Function
cat("\nUsing D() function:\n")
print(D(quote(sinpi(x^2)), "x"))
```

Output:

```
Using deriv() function:
expression({
    .expr1 <- x^2
    .value <- sinpi(.expr1)
    .grad <- array(0, c(length(.value), 1L), list(NULL, c("x")))
    .grad[, "x"] <- cospi(.expr1) * (pi * (2 * x))
    attr(.value, "gradient") <- .grad
    .value
})

Using D() function:
cospi(x^2) * (pi * (2 * x))
```

Integral Calculus

The analysis of integrals and their properties is known as integral calculus. It is primarily useful for the following two functions: To compute f from f' (i.e. from its derivative). If a function f is differentiable in the range under consideration, then f' is specified in that range. To determine the region under a curve. **Differentiation** is the inverse of **integration**. As separation can be defined as the division of a part into several small parts, integration can be defined as the selection of small parts to form a whole. It is commonly used to calculate area.

A **definite integral** has a specified boundary beyond which the equation must be computed. The lower and upper limits of a function's independent variable are defined, and its integration is represented using definite integrals. An **infinite integral** lacks a fixed boundary, i.e. there is no upper and lower limit. As a result,

Question 2: Find the integral for the given function f(x),

$$f(x) = 2x^2 + 3x$$

Solution:

Given $f(x) = 2x^2 + 3x$

$2x^2 + 3x$ is a polynomial function, and it can be broken down in different parts.

$\int f(x)dx$

$\Rightarrow \int (2x^2 + 3x)dx$

Using the property 1 mentioned above,

$\int 2x^2 dx + \int 3x dx$

$\frac{x^3}{3} + \frac{3x^2}{2} + C$

Applications of Calculus

- Examining a system to discover the best approach to forecast any given circumstance for a function.
- Calculus concepts are widely used in everyday life, whether it is to solve problems with complex shapes, assess survey results, determine the safety of automobiles, design a business, track credit card payments, or determine how a system is developing and how it affects us, etc.
- Economists, biologists, architects, doctors, and statisticians all speak calculus. For instance, engineers and architects employ several calculus ideas to determine the size and design of construction structures.
- Modeling ideas like occurrence and mortality rates, radioactive decay, reaction rates, heat and light, motion, and electricity all employ calculus.

4.3.5 Functions for Statistical Distributions

77 xz3 that R handles has four functions. There is a root name, for example, the root name for the normal distribution is norm. This root is prefixed by one of the letters

- **p** for "probability", the cumulative distribution function (c. d. f.)
- **q** for "quantile", the inverse c. d. f.
- **d** for "density", the density function (p. f. or p. d. f.)
- **r** for "random", a random variable having the specified distribution

For the normal distribution, these functions are pnorm, qnorm, dnorm, and rnorm. For the binomial distribution, these functions are pbinom, qbinom, dbinom, and rbinom. And so forth.

For a continuous distribution (like the normal), the most useful functions for doing problems involving probability calculations are the "p" and "q" functions (c. d. f. and inverse c. d. f.), because the the density (p. d. f.) calculated by the "d" function can only be used to calculate probabilities via integrals and R doesn't do integrals.

For a discrete distribution (like the binomial), the "d" function calculates the density (p. f.), which in this case is a probability

f(x) = P(X = x)

and hence is useful in calculating probabilities.

R has functions to handle many probability distributions. The table below gives the names of the functions for each distribution and a link to the on-line documentation that is the authoritative reference for how the functions are used. But don't read the on-line documentation yet. First, try the examples in the sections following the table.

Beta	pbeta	qbeta	dbeta	rbeta
Binomial	pbinom	qbinom	dbinom	rbinom
Cauchy	pcauchy	qcauchy	dcauchy	rcauchy
Chi-Square	pchisq	qchisq	dchisq	rchisq
Exponential	Pexp	qexp	dexp	rexp

F	Pf	qf	df	rf
Gamma	pgamma	qgamma	dgamma	rgamma
Geometric	pgeom	qgeom	dgeom	rgeom
Hypergeometric	phyper	qhyper	dhyper	rhyper
Logistic	plogis	qlogis	dlogis	rlogis
Log Normal	plnorm	qlnorm	dlnorm	rlnorm
Negative Binomial	pnbinom	qnbinom	dnbinom	rnbinom
Normal	pnorm	qnorm	dnorm	rnorm
Poisson	Ppois	qpois	dpois	rpois
Student t	Pt	qt	dt	rt
Studentized Range	ptukey	qtukey	dtukey	rtukey
Uniform	punif	qunif	dunif	runif
Weibull	pweibull	qweibull	dweibull	rweibull
Wilcoxon Rank Sum Statistic	pwilcox	qwilcox	dwilcox	rwilcox
Wilcoxon Signed Rank Statistic	psignrank	qsignrank	dsignrank	rsignrank

They look up P(X < 27.4) when X is normal with mean 50 and standard deviation 20.

UNIT V

OBJECT-ORIENTED PROGRAMMING

S Classes

Class System in R

While most programming languages have a single class system, R has three class systems:

S3 Class

S4 Class

Reference Class

The original R structure for classes, known as S3, is still the dominant class paradigm in R use today. Indeed, most of R's own built-in classes are of the S3 type.
An S3 class consists of a list, with a class name attribute and dispatch capability added.

S3 Class in R

S3 class is the most popular class in the R programming language. Most of the classes that come predefined in R are of this type.
First we create a list with various components then we create a class using the class() function. For example,

```
# create a list with required components
student1 <- list(name = "John", age = 21, GPA = 3.5)

# name the class appropriately
class(student1) <- "Student_Info"

# create and call an object
student1
```

Output

```
$name
[1] "John"

$age
[1] 21

$GPA
[1] 3.5

attr(,"class")
[1] "student"
```

Activate Windows

In the above example, we have created a list named student1 with three components. Notice the creation of class,

```
class(student1) <- "Student_Info"
```

Here, Student_Info is the name of the class. And to create an object of this class, we have passed the student1 list inside class().

Finally, we have created an object of the Student_Info class and called the object student1.

S4 Class in R

S4 class is an improvement over the S3 class. They have a formally defined structure which helps in making objects of the same class look more or less similar.

In R, we use the setClass() function to define a class. For example,

Example: S4 Class in R

```
# create a class "Student_Info" with three member variables
setClass("Student_Info", slots=list(name="character", age="numeric", GPA="numeric"))

# create an object of class
student1 <- new("Student_Info", name = "John", age = 21, GPA = 3.5)

# call student1 object
student1
```

Output

```
An object of class "Student_Info"
Slot "name":
[1] "John"

Slot "age":
[1] 21

Slot "GPA":
[1] 3.5
```

Here, we have created an S4 class named `Student_Info` using the `setClass()` function and an object named `student1` using the `new()` function.

Reference Class in R

Reference classes were introduced later, compared to the other two. It is more similar to the object oriented programming we are used to seeing in other major programming languages.

Defining a reference class is similar to defining a S4 class. Instead of setClass() we use the setRefClass() function. For example,

```
# create a class "Student_Info" with three member variables
Student_Info <- setRefClass("Student_Info",
  fields = list(name = "character", age = "numeric", GPA = "numeric"))

# Student_Info() is our generator function which can be used to create new objects
student1 <- Student_Info(name = "John", age = 21, GPA = 3.5)

# call student1 object
student1
```

Output

```
Reference class object of class "Student_Info"
Field "name":
[1] "John"
Field "age":
[1] 21
Field "GPA":
[1] 3.5
```

Generic Functions

R is polymorphic, in the sense that the same function can lead to different operations for different classes. You can apply plot(), for example, to many different types of objects, getting a different type of plot for each. The same is true for print(), summary(), and many other functions.

In this manner, we get a uniform interface to different classes. For example, if you are writing code that includes plot operations, polymorphism may allow you to write your program without worrying about the various types of objects that might be plotted.

In addition, polymorphism certainly makes things easier to remember for the user and makes it fun and convenient to explore new library functions and associated classes. If a function is new to you, just try running plot() on the function's output; it will likely work. From a programmer's viewpoint, polymorphism allows writing fairly general code, without worrying about what type of object is being manipulated, because the underlying class mechanisms take care of that.

The functions that work with polymorphism, such as plot() and print(), are known as generic functions. When a generic function is called, R will then dispatch the call to the proper class method, meaning that it will reroute the call to a function defined for the object's class.

Example: OOP in the lm() Linear Model Function

As an example, let's look at a simple regression analysis run via R's lm() function.

First, let's see what lm() does:

```
> ?lm
```

The output of this help query will tell you, among other things, that this function returns an object of class "lm".

Let's try creating an instance of this object and then printing it:

```
> x <- c(1,2,3)
> y <- c(1,3,8)
> lmout <- lm(y ~ x)
> class(lmout)
[1] "lm"
> lmout

Call:
lm(formula = y ~ x)

Coefficients:
(Intercept)            x
       -3.0          3.5
```

Here, we printed out the object lmout. (Remember that by simply typing the name of an object in interactive mode, the object is printed.) The R interpreter then saw that lmout was an object of class "lm" and thus called print.lm(), a special print method for the "lm" class. In R terminology, the call to the generic function print() was dispatched to the method

print.lm() associated with the class "lm".Let's take a look at the generic function and the class method in this case:

```
> print
function(x, ...) UseMethod("print")
<environment: namespace:base>
> print.lm
function (x, digits = max(3, getOption("digits") - 3), ...)
{
    cat("\nCall:\n", deparse(x$call), "\n\n", sep = "")
    if (length(coef(x))) {
        cat("Coefficients:\n")
        print.default(format(coef(x), digits = digits), print.gap = 2,
            quote = FALSE)
    }
    else cat("No coefficients\n")
    cat("\n")
    invisible(x)
}
<environment: namespace:stats>
```

You may be surprised to see that print() consists solely of a call to UseMethod(). But this is actually the dispatcher function, so in view of print()'s role as a generic function, you should not be surprised after all.

Writing S3 Classes

S3 classes have a rather cobbled-together structure. A class instance is created by forming a list, with the components of the list being the member variables of the class. The "class" attribute is set by hand by using the attr() or class() function, and then various implementations of generic functions are defined. We can see this in the case of lm() by inspecting the function:

```
> lm
...
z <- list(coefficients = if (is.matrix(y))
                    matrix(,0,3) else numeric(0L), residuals = y,
                  fitted.values = 0 * y, weights = w, rank = 0L,
                  df.residual = if (is.matrix(y)) nrow(y) else length(y))
}
...
class(z) <- c(if(is.matrix(y)) "mlm", "lm")
...
```

Using Inheritance

The idea of inheritance is to form new classes as specialized versions of old ones. In our previous employee example, for instance, we could form a new class devoted to hourly employees, "hrlyemployee", as a subclass of "employee", as follows:

```
k <- list(name="Kate", salary= 68000, union=F, hrsthismonth= 2)
class(k) <- c("hrlyemployee","employee")
```

Our new class has one extra variable: hrsthismonth. The name of the new class consists of two character strings, representing the new class and the old class. Our new class inherits the methods of the old one. For instance, print.employee() still works on the new class:

```
> k
Kate
salary 68000
union member FALSE
```

Once again, simply typing k resulted in the call print(k). In turn, that caused UseMethod() to search for a print method on the first of k's two class names, "hrlyemployee". That search failed, so UseMethod() tried the other class name, "employee", and found print.employee(). It executed the latter. Recall that in inspecting the code for "lm", you saw this line:

```
class(z) <- c(if(is.matrix(y)) "mlm", "lm")
```

Implementing a Generic Function on an S Class

To define an implementation of a generic function on an S4 class, use setMethod(). Let's do that for our class "employee" here. We'll implement the show() function, which is the S4 analog of S3's generic "print". As you know, in R, when you type the name of a variable while in interactive mode, the value of the variable is printed out:

```
> joe
An object of class "employee"
Slot "name":
[1] "Joe"

Slot "salary":
[1] 88000

Slot "union":
[1] TRUE
```

Since joe is an S4 object, the action here is that show() is called. In fact, we would get the same output by typing this:

```
> show(joe)
```

Let's override that, with the following code:

```
setMethod("show", "employee",
   function(object) {
      inorout <- ifelse(object@union,"is","is not")
      cat(object@name,"has a salary of",object@salary,
         "and",inorout, "in the union", "\n")
   }
)
```

The first argument gives the name of the generic function for which we will define a class-specific method, and the second argument gives the class name. We then define the new function.

S3 Vs S4

The S3 and S4 software in R are two generations implementing functional object-oriented programming. S3 is the original, simpler for initial programming but less general, less formal and less open to validation. The S4 formal methods and classes provide these features but require more programming.

Visualization

Data visualization is the technique used to deliver insights in data using visual cues such as graphs, charts, maps, and many others. This is useful as it helps in intuitive and easy understanding of the large quantities of data and thereby make better decisions regarding it.

Data Visualization in R Programming Language

The popular data visualization tools that are available are Tableau, Plotly, R, Google Charts, Infogram, and Kibana. The various data visualization platforms have different capabilities, functionality, and use cases. They also require a different skill set. This article discusses the use of R for data visualization.

R is a language that is designed for statistical computing, graphical data analysis, and scientific research. It is usually preferred for data visualization as it offers flexibility and minimum required coding through its packages.

Types of Data Visualizations

Some of the various types of visualizations offered by R are:

Bar Plot

There are two types of bar plots- horizontal and vertical which represent data points as horizontal or vertical bars of certain lengths proportional to the value of the data item. They are generally used for continuous and categorical variable plotting. By setting the **horiz** parameter to true and false, we can get horizontal and vertical bar plots respectively.

Bar plots are used for the following scenarios:

1.To perform a comparative study between the various data categories in the data set.

2.To analyze the change of a variable over time in months or years.

Histogram

A histogram is like a bar chart as it uses bars of varying height to represent data distribution. However, in a histogram values are grouped into consecutive intervals called bins. In a Histogram, continuous values are grouped and displayed in these bins whose size can be varied.

For a histogram, the parameter **xlim** can be used to specify the interval within which all values are to be displayed. Another parameter **freq** when set to TRUE denotes the frequency of the various values in the histogram and when set to FALSE, the probability densities are represented on the y-axis such that they are of the histogram adds up to one.

Histograms are used in the following scenarios:

- To verify an equal and symmetric distribution of the data.
- To identify deviations from expected values.

Box Plot

The statistical summary of the given data is presented graphically using a boxplot. A boxplot depicts information like the minimum and maximum data point, the median value, first and third quartile, and interquartile range.

Box Plots are used for:

- To give a comprehensive statistical description of the data through a visual cue.
- To identify the outlier points that do not lie in the inter-quartile range of data.

Scatter Plot

A scatter plot is composed of many points on a Cartesian plane. Each point denotes the value taken by two parameters and helps us easily identify the relationship between them.

Scatter Plots are used in the following scenarios:

- To show whether an association exists between bivariate data.

- To measure the strength and direction of such a relationship.

Heat Map

Heatmap is defined as a graphical representation of data using colors to visualize the value of the matrix. heatmap() function is used to plot heatmap.

Advantages of Data Visualization in R:

R has the following advantages over other tools for data visualization:

R offers a broad collection of visualization libraries along with extensive online guidance on their usage.

R also offers data visualization in the form of 3D models and multipanel charts.

Through R, we can easily customize our data visualization by changing axes, fonts, legends, annotations, and labels.

Disadvantages of Data Visualization in R:

R also has the following disadvantages:

R is only preferred for data visualization when done on an individual standalone server.

Data visualization using R is slow for large amounts of data as compared to other counterparts.

Application Areas:

Presenting analytical conclusions of the data to the non-analysts departments of your company.

Health monitoring devices use data visualization to track any anomaly in blood pressure, cholesterol and others.

To discover repeating patterns and trends in consumer and marketing data.

Meteorologists use data visualization for assessing prevalent weather changes throughout the world.

Simulation

Simulations are a powerful statistical tool. Simulation techniques allow us to carry out statistical inference in complex models, estimate quantities that we can cannot calculate analytically or even to predict under different scenarios the outcome of some scenario such as an epidemic outbreak. In this section, we will cover the basics of simulations and simulation experiments. It will cover

- simulating a sample from common probability distributions
- simulation experiments for sampling distributions
- simulation experiments for type I error rates and power calculations.

Standard Probability Distributions

Sometimes you want to generate data from a distribution (such as normal), or want to see where a value falls in a known distribution. R has these distributions built in:

Sampling from More Complex Distributions

There are many techniques that have been developed to sample from complex probability distributions. Some of these are used to generate samples from the r functions. Basic introduction to two techniques: the accept-reject algorithm and the Metropolis-Hastings Markov Chain Monte Carlo Algorithm (MCMC) algorithm.

The Accept-Reject Algorithm

The accept-reject algorithm is a method of generating a random sample from a probability distribution by first generating a proposal sample from an "envelope" distribution, which is easy to sample from, and then deciding whether or not to accept or reject this sample.

To motivate the rejection method, let us consider a simple example. say we have a continuous random variable X with pdf fX concentrated on the interval (0,2) as shown below. We imagine "sprinkling" points P1,P2,…1,2,…, uniformly at random under the density function. By sprinkling uniformly, we mean that a small target square under the pdf has the same chance of being hit wherever it is located. Our random points, Pt, are actually two-dimensional random variables (Xi,Yi), where Xi and Yi are the random coordinates of the i-th coordinate.

How do we get these random points? First we simulate uniformly under some "envelope" region. Since we can't simulate directly from fX, let's consider simulating from another "envelope" distribution with density h*h* that we can simulate from. In our example above, h=0.5,0<x<2,*h*=0.5,0<x<2, is the density of the uniform distribution on (0,2). If we then let k be such that kh≥fX and Y∼U(0,kh(X)), given X, then (X,Y) will be uniformly distributed over the region defined by the area below the curve k*h*. In our example above, k=2. But we only want the points under the true density, so we accept X values if Y<fX(X) and reject them otherwise.

In this case,for the accepted X

$$
\begin{aligned}
P(a < X < b) &= \int_0^{\infty} P(a < X < b, Y \leq y)dy \\
&= \int_{-0}^{\infty} \int_a^b P(Y \leq y|X = x) f_X(x) dx dy \\
&= \int_a^b \int_{-0}^{fx(x)} P(Y \leq y|X = x) f_X(x) dy dx \quad (\text{since } Y|X, Accept \sim U(0, f_X(X))) \\
&= \int_a^b \int_{-0}^{fx(x)} P(Y \leq y|X = x) f_X(x) dy dx \\
&= \int_a^b f_X(x) dx
\end{aligned}
$$

The Metropolis-Hastings Alogorithm

```
accept_reject = function(fx, n = 100) {
  #simulates a sample of size n from the pdf fx using the accept reject algorithm
  x = numeric(n)
  count = 0

  while(count < n) {
    temp <- runif(1, 0, 2)
    y <- runif(1, 0, 2)
    if (y < fx(temp)) {
      count = count + 1
      x[count] <- temp
    }
  }

  return(x)
}

sample = accept_reject(triangle.pdf, 10000)
ggplot(triangle_plot.dat, aes(x = x, y = y)) +
  geom_line(color = "blue", size = 1.5) + xlab("x") + ylab("pdf") +
  geom_histogram(data = data.frame(x = sample), aes(x = x, y = ..density..), col = "gray" )
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

Sometimes the distribution is too complicated to find an envelope function to use the accept reject algorithm. This is very common in Bayesian models. Another option to generate a sample from a complex distribution is Markov chain Monte Carlo (MCMC). In

this section we will look at an example of the Metropolis-Hastings algorithm, which is one of many MCMC algorithms.

The MCMC algorithm generates a markov chain X1,X2,…1,2,… by using a series of draws from a more common distribution, choosing at random which of these proposed draws to accept as draws from the target distribution. The probability of acceptance is calculated so that after the process has converged the accepted draws form a sample from the desired distribution.

Let's consider the distribution with density

$$f(y) = c\exp(-y^4)(1+|y|)^3$$

where c is a normalizing constant and $-\infty < y < \infty$. This was considered by Evans and Rosenthal

where c is a normalizing constant and −∞<y<∞. This was considered by Evans and Rosenthal (2004, Probability and Statistic).

The algorithm uses a proposal density q(x,y)=q(y|x) and decides whether or not to accept this proposal with probability α(x,t). For the proposal density, we try to pick a density that is close to f(y) and easy to simulate from. We will use the normal distribution with mean equal to the previous accepted value, x, and variance 1. The acceptance probability is then

$$\begin{aligned}
\alpha(x,y) &= \min\left\{1, \frac{f(y)q(y,x)}{f(x)q(x,y)}\right\} \\
&= \min\left\{1, \frac{c\exp(-y^4)(1+|y|)^3(2\pi)^{-1/2}\exp(-(y-x)^2/2)}{c\exp(-x^4)(1+|x|)^3(2\pi)^{-1/2}\exp(-(x-y)^2/2)}\right\} \\
&= \min\left\{1, \frac{\exp(-y^4+x^4)(1+|y|)^3}{(1+|x|)^3}\right\}
\end{aligned}$$

To begin the process, we pick an arbitrary value for X_1 to start from. Then the Metropolis-Hastings algorithm chooses X_{n+1} as follows:

1. Generate y from a Normal(X_{n}, 1)$
2. Compute $\alpha(X_n, y)$ (formula given above)
3. With probability $\alpha(X_n, y)$, let $X_{n+1} = y$ (accept new value). Otherwise, with probability $1 - \alpha(X_n, y)$, let $X_{n+1} = X_n$ (keep the previous value).

In the code below, we have a burnin period, where we discard the initial observations to allow the markov chain time to get close to stationarity (the point where iterations start to

come from the desired distribution), and we thin (only keep every 10th value) to reduce the correlation between our generated data.

```
alphafun = function(x, y) {
  exp(-y^4 + x^4)*(1+abs(y))^3*(1+abs(x))^{-3}
}

mh_mcmc = function(alphafn, burnin, N, thin) {
  x = numeric(burnin + N*thin)
  x[1] = rnorm(1)
  for (i in 2:(burnin + N*thin)) {
    propy = rnorm(1, x[i - 1], 1)
    alpha = min(1, alphafn(x[i-1], propy))
    x[i] = ifelse(rbinom(1, 1, alpha), propy, x[i-1])
  }
  return(x[burnin + (1:N)*thin])
}

sample = mh_mcmc(alphafun, 5000, 10000, 10)

f_true = function(x) {
  exp(-x^4)*(1+abs(x))^3
}

integral = integrate(f_true, 0, Inf)
c = 2 * integral$value
```

```
f_true = function(x) {
  exp(-x^4)*(1+abs(x))^3 / c
}

f_true_plot.dat = data.frame(x = seq(-2, 2, length = 100),
                             y = f_true(seq(-2, 2, length = 100)))

ggplot(f_true_plot.dat, aes(x, y)) +
  geom_line(color = "blue", size = 1.2) + ylab("pdf") +
  geom_histogram(data = data.frame(x = sample), aes(x = x, y = ..density..),
                 color = "gray")
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

Simulation Studies/Experiments

What is a simulation study?

- A numerical technique for conducting experiments on a computer
- It involves randomly sampling from probability distributions

  Why conduct a simulation study?

- To validate a statistical method so people can use it with confidence
- Examine analytic properties that are rarely possible to calculate exactly
- Check how large N properties behave in (finite) samples
- Check how a statistical technique performs when the assumptions are not met

Simulating an Epidemic from a SIR Model

- The science of epidemiology, the study of the spread of disease, includes mathematical/statistical models of how disease spreads. In the SIR model, which stands for Susceptible, Infected, and Removed, we suppose that individuals can be one of three.
- Let S(t), I(t), and R(t) be the number of susceptible, infected and removed individuals at time t. At each time step, each infected has probability α of infecting each susceptible. (This assumes that each infected has equal contact with all susceptibles. This is called a mixing assumption.) At the end of each time step, after having had a chance to infect people, each infected has probability β of being removed.

Code Profiling

Code profiling tools allows to analyze the performance of the code by measuring the time it takes the methods to run and the amount of CPU and memory they consume.

Characteristic Features:

1.Discover and optimize bottlenecks in your code

- Detect high resource-consuming methods and optimize application performance with code profiling tools.
- Gain visibility into CPU and memory consumption, as well as time spent on locks, I/O, and garbage collection, down to the line of code
- Reduce latency, improve end-user experience, and save on cloud provider costs by optimizing your slowest and resource-heavy lines of code

2. Always on, production level code profiling tools

- Reduce MTTR by pinpointing production code issues that are invisible to other tools and hard to replicate in other environments
- Explore multiple profile types—CPU, memory, lock, I/O, and more—to determine the root cause of code issues
- Derive actionable code profiling insights from an automated heuristic analysis of the main problem areas in your code

3. Correlate code profiles with all other telemetry

- Tie every slow distributed trace to the methods and threads that executed the request
- Quickly detect and resolve anomalous spikes in infrastructure metrics caused by inefficiencies in your code
- Compare code behavior and impact across hosts, services, and versions during code deployments

Statistical Analysis with R

Statistical Analysis with R is one of the best practices which the statistician, data analysts, and data scientists do while analyzing statistical data. R language is a popular open-source programming language that extensively supports built-in packages and external packages for statistical analysis. R language natively supports basic statistical calculations for exploratory data, and advanced statistics for predictive data analysis Statistical analysis with R is an important part of identifying data patterns based upon the statistical rules and business constraints. Due to the simplicity of R syntax and flexibility of using advanced packages. R language is preferred for Statistical Analysis.

R is a freely distributed software package for statistical analysis and graphics, developed and managed by the R Development Core Team. R can be downloaded from the Internet site of the Comprehensive R Archive Network (CRAN) (http://cran.r-project.org). Check that you download the correct version of R for your operating system (for example, XP for the PC, Tiger or earlier versions of OSX for Macs). R is related to the S statistical language which is commercially available as S-PLUS.

R is an object-oriented language. For our basic applications, matrices representing data sets (where columns represent different variables and rows represent different subjects) and column vectors representing variables (one value for each subject in a sample) are objects in R. Functions in R perform calculations on objects. For example, if 'cholesterol' was an object representing cholesterol levels from a sample, the function 'mean(cholesterol)' would calculate the mean cholesterol for the sample. For our basic applications, results of an analysis are displayed on the screen. Results from analyses can also be saved as objects in R, allowing the user to manipulate results or use the results in further analyses.

Data can be directly entered into R, but we will usually use MS Excel to create a data set. Data sets are arranged with each column representing a variable, and each row representing a subject; a data set with 5 variables recorded on 50 subjects would be represented in an Excel file with 5 columns and 50 rows. Data can be entered and edited using Excel. Excel can save files in 'comma delimited format', or .csv files; these .csv files can then be read into R for analysis.

R is an interactive language.

Computing Average in R Programming

To compute the average of values, R provides a pre-defined function **mean()**. This function takes a Numerical Vector as an argument and results in the average/mean of that Vector.

Syntax: mean(x, na.rm)

Parameters:

- **x:** Numeric Vector
- **na.rm:** Boolean value to ignore NA value

```r
# R program to get average of a list

# Taking a list of elements
list = c(2, 4, 4, 4, 5, 5, 7, 9)

# Calculating average using mean()
print(mean(list))
```

Variance in R Programming Language

Variance is the sum of squares of differences between all numbers and means. The mathematical formula for variance is as follows, Standard Deviation is the square root of variance. It is a measure of the extent to which data varies from the mean. The mathematical formula for calculating standard deviation is as follows,

Example:

Standard Deviation for the above data,

$StandardDeviation = \sqrt{4} = 2$

Computing Standard Deviation in R

One can calculate the standard deviation by using sd() function in R.

Syntax: *sd(x)*

Parameters:

x: *numeric vector*

Mean, Median and Mode in R Programming

The **measure of central tendency** in R Language represents the whole set of data by a single value. It gives us the location of central points. There are three main measures of central tendency:

- Mean
- Median
- Mode

the sorted values.

Using Modeest Package

We can use the modeest package of the R. This package provides methods to find the mode of the univariate data and the mode of the usual probability distribution.

Example:

R

```
# R program to illustrate
# Descriptive Analysis

# Import the library
library(modeest)

# Import the data using read.csv()
myData = read.csv("CardioGoodFitness.csv",
                  stringsAsFactors=F)

# Compute the mode value
mode = mfv(myData$Age)
print(mode)
```

Output:

```
[1] 25
```

Data Manipulation

Data manipulation involves modifying data to make it easier to read and to be more organized. We manipulate data for analysis and visualization. It is also used with the term ‘data exploration’ which involves organizing data using available sets of variables. At times, the data collection process done by machines involves a lot of errors and

inaccuracies in reading. Data manipulation is also used to remove these inaccuracies and make data more accurate and precise.

Data Manipulation in R With dplyr Package

There are different ways to perform data manipulation in R, such as using **Base R functions** like subset(), with(), within(), etc., **Packages** like data.table, ggplot2, reshape2, readr, etc., and different Machine Learning algorithms. However, in this tutorial, we are going to use the **dplyr** package to perform data manipulationinR.

The **dplyr** package consists of many functions specifically used for data manipulation. These functions process data faster than Base R functions and are known the best for data exploration and transformation, as well. Following are some of the important functions included in the **dplyr** package

filter()	Produces a subset of a Data Frame.
distinct()	Removes duplicate rows in a Data Frame
arrange()	Reorder the rows of a Data Frame
select()	Produces data in required columns of a Data Frame
rename()	Renames the variable names
mutate()	Creates new variables without dropping old ones.
transmute()	Creates new variables by dropping the old.
summarize()	Gives summarized data like Average, Sum, etc.

00 runs from the “stats” data frame.

Here in this example, we used distinct() method to remove the duplicate rows from the data frame and also remove duplicates based on a specified column.

```
# import dplyr package
library(dplyr)

# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D', 'A', 'A'),
                    runs=c(100, 200, 408, 19, 56, 100),
                    wickets=c(17, 20, NA, 5, 2, 17))

# removes duplicate rows
distinct(stats)

#remove duplicates based on a column
distinct(stats, player, .keep_all = TRUE)
```

Output

```
  player runs wickets
1      A  100      17
2      B  200      20
3      C  408      NA
4      D   19       5
5      A   56       2
  player runs wickets
1      A  100      17
2      B  200      20
3      C  408      NA
4      D   19       5
```

arrange() method

In R, the arrange() method is used to order the rows based on a specified column. The syntax of arrange() method is specified below-

arrange(dataframeName, columnName)

Example:
In the below code we ordered the data based on the runs from low to high using arrange() function.

select() method

The select() method is used to extract the required columns as a table by specifying the required column names in select() method. The syntax of select() method is mentioned below-

select(dataframeName, col1,col2,...)

Example:
Here in the below code we fetched the player, wickets column data only using select() method.

```r
# import dplyr package
library(dplyr)

# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D'),
                    runs=c(100, 200, 408, 19),
                    wickets=c(17, 20, NA, 5))

# fetch required column data
select(stats, player,wickets)
```

Output

```
  player wickets
1      A      17
2      B      20
3      C      NA
4      D       5
```

rename() method

The rename() function is used to change the column names. This can be done by the below syntax-

rename(dataframeName, newName=oldName)

Example:

In this example, we change the column name “runs” to “runs_scored” in stats data frame.

```r
# import dplyr package
library(dplyr)

# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D'),
                    runs=c(100, 200, 408, 19),
                    wickets=c(17, 20, NA, 5))

# renaming the column
rename(stats, runs_scored=runs)
```

Output

```
  player runs_scored wickets
1      A         100      17
2      B         200      20
3      C         408      NA
4      D          19       5
```

mutate() & transmute() methods

These methods are used to create new variables. The mutate() function creates new variables without dropping the old ones but transmute() function drops the old variables and creates new variables. The syntax of both methods is mentioned below-

mutate(dataframeName, newVariable=formula)

transmute(dataframeName, newVariable=formula)

summarize() method

Using the summarize method we can summarize the data in the data frame by using aggregate functions like sum(), mean(), etc. The syntax of summarize() method is specified below-

summarize(dataframeName, aggregate_function(columnName))

Example:

In the below code we presented the summarized data present in the runs column using summarize() method.

```r
# import dplyr package
library(dplyr)

# create a data frame
stats <- data.frame(player=c('A', 'B', 'C', 'D'),
                    runs=c(100, 200, 408, 19),
                    wickets=c(17, 20, 7, 5))

# summarize method
summarize(stats, sum(runs), mean(runs))
```

Output

```
  sum(runs) mean(runs)
1       727     181.75
```